NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Computer Networks

Redes de Computadores

RC

Teórica OITO

LEI

João Leitão

©2026 NOVA FCT

Aula Oito

- Sondas de Janela Zero e Janelas Grandes
- Decidir Quando Enviar
- Retransmissão Adaptativa
- Controlo de Saturação: Porquê, e O Quê
- Aumento Aditivo, Diminuição Multiplicativa
- Slow Start
- Fast Retransmit e Fast Recovery

A Semana Passada

- **As ligações têm um ciclo de vida:** um three-way handshake abre-as; uma troca de quatro mensagens FIN/ACK (ou, abruptamente, um único RST) fecha-as.
- **O controlo de fluxo reutiliza a janela deslizante:** o recetor anuncia o seu espaço livre em buffer, e o emissor nunca tem mais do que isso em trânsito.
- **Ficámos com um problema em aberto:** o buffer do recetor cheio, AdvertisedWindow = 0, e o emissor mandado parar por completo.
- **Hoje:** o resto dos mecanismos principais do TCP — e depois um problema novo: como ser “simpático” para a rede (*network friendly*).

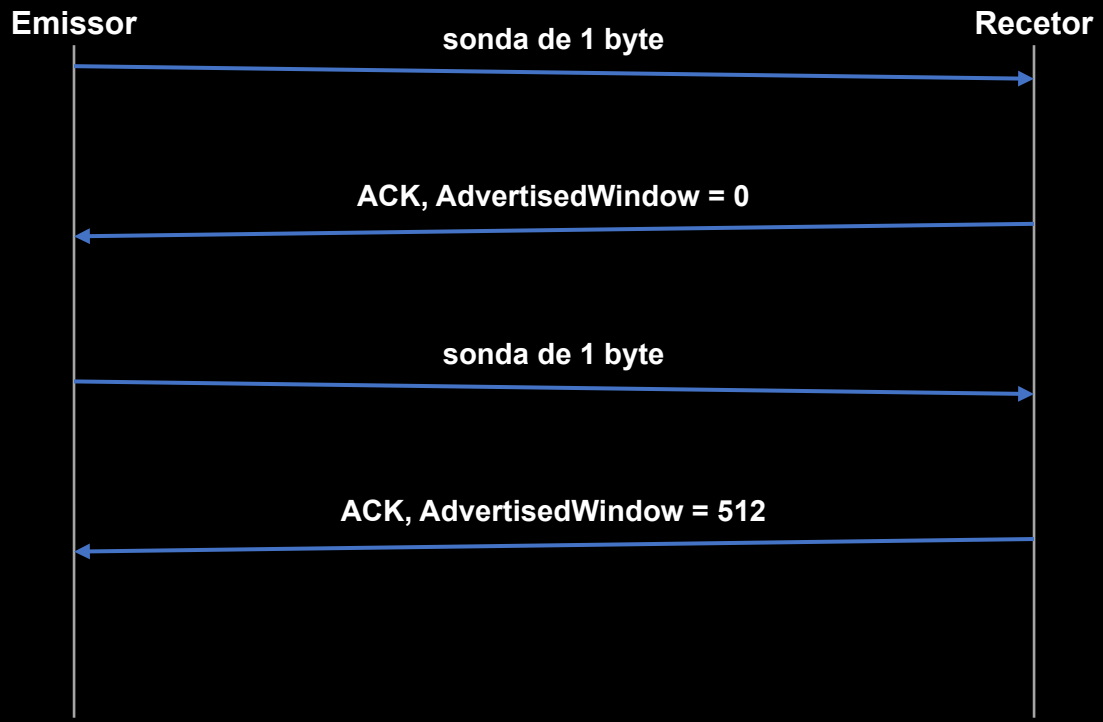
O Que Acontece em Zero?

- Uma AdvertisedWindow anunciada como 0 no recetor significa que o emissor não tem mais nada que lhe seja permitido enviar. Mas o TCP nunca envia dados sem ser solicitado — então como é que o emissor descobre que a janela do recetor voltou a abrir?

Sondas de Janela Zero

- **O emissor continua a enviar 1 byte de dados reais a cada 5–60 segundos;** cada sonda obriga a uma resposta que transporta o AdvertisedWindow atual.

Sondas de Janela Zero



Manter a Ligação Cheia

- **O AdvertisedWindow de 16 bits limita a janela a 64 KB** — perfeitamente adequado para uma ligação dial-up, desesperadamente pequeno para uma rápida.
  - Com um RTT de 100 ms, uma ligação de 10 Gbps precisa de cerca de 125 MB em trânsito para se manter cheia — longe dos 64 KB.

- **A solução:** uma extensão do TCP (window scaling) alarga efetivamente este campo.

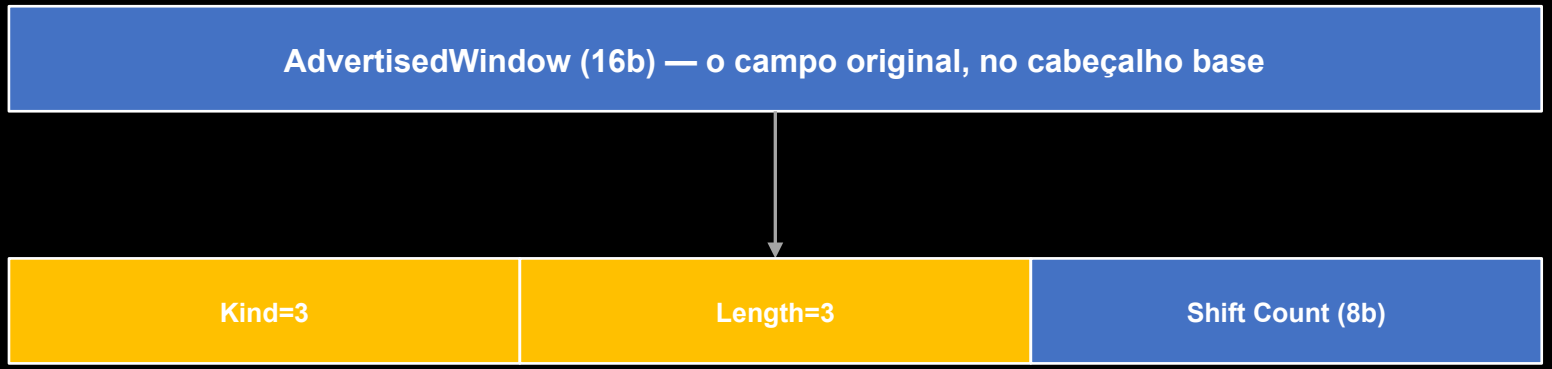
Proteger Contra o Espaço Circular

- **O SequenceNum de 32 bits pode dar a volta e repetir-se** — o mesmo número reutilizado para um byte diferente, confundindo o algoritmo da janela deslizante.
  - A 10 Gbps, o espaço de 32bits dá a volta completa em cerca de 3 segundos — o mesmo pressuposto de tempo de vida do pacote (MSL ≈ 120s) que justificava o TIME_WAIT já não nos protege a estas velocidades.
- Mais uma razão para o TCP de alta velocidade precisar de extensões: a opção Timestamp alarga efetivamente o espaço de sequência, para que bytes antigos e novos com o mesmo número possam ser distinguidos.

Uma Opção, Não um Campo Novo

- **As extensões reais viajam como Options, não como campos novos —** e o window scaling é uma delas — por isso o HdrLen é variável: o espaço extra existe para isto.
- **A opção Window Scale é acordada uma única vez, durante o handshake:** cada lado anuncia, no seu SYN, o seu próprio shift count para as janelas que vai anunciar.
- **Um shift de 11** transforma cada unidade desse campo de 1 byte em 2048 bytes, esticando o limite de 64 KB para cerca de 134 MB (128 MiB).
- **O cabeçalho base em si nunca muda —** o AdvertisedWindow continua a ser os mesmos 16 bits que sempre foi; só a sua unidade de medida cresce.

Ampliando as Options



Enviado uma vez por cada lado, no seu SYN: os valores de AdvertisedWindow que esse lado anuncia são deslocados à esquerda o seu Shift Count antes de serem usados — por exemplo, 11 transforma o limite de 64 KB em cerca de 134 MB (128 MiB).

Controlo de Fluxo, Não Controlo de Saturação

Conclusão:
O AdvertisedWindow protege o recetor do emissor — controlo de fluxo. Não diz nada sobre proteger a própria rede de ficar sobrecarregada — isso é controlo de saturação, um problema diferente, que vem mais à frente nesta aula.

Uma Questão não Trivial

- Já sabemos como é um segmento TCP, e como a janela limita quantos dados podem estar em trânsito. Mas há uma questão que fomos silenciosamente saltando: dada uma abstração de fluxo de bytes, quando é que o TCP decide enviar um segmento?

Três possibilidades para Enviar

- **Dados suficientes para um segmento completo:** o TCP espera até ter reunido MSS (Maximum Segment Size) bytes (normalmente cerca de 1460 bytes), e só depois envia — o caso comum quando uma ligação já está em velocidade de cruzeiro.

- **Um push explícito:** a aplicação pede ao TCP para esvaziar imediatamente o que estiver em buffer, mesmo que seja menos do que um segmento completo.

- **Um temporizador:** se nenhuma das duas situações acima acontecer, o TCP acaba por enviar o que tiver — mas definir o valor deste “temporizador” acaba por ser mais complicado do que parece.

E Se a Janela Estiver Quase Fechada?

- Suponham que o emissor tem dados para enviar, e a janela está aberta — mas só um pouco (ou seja, o recetor só tem espaço para uns poucos bytes). Deve enviar já um segmento minúsculo, ou esperar que se abra mais espaço?

A Silly Window Syndrome

- **Enviar agressivamente assim que a janela abre** leva a uma patologia descoberta nas primeiras implementações do TCP: um fluxo de segmentos minúsculos, dominados sobretudo por overhead, cada um transportando muitos poucos bytes de dados.
  - Pensem num tapete rolante construído para contentores grandes a transportar de repente um pacotinho de cada vez — o tapete nunca para, mas quase não move carga real nenhuma.

- **Assim que um segmento pequeno entra no sistema,** a ligação tende a só transmitir segmentos minúsculos: o recetor confirma exatamente aquele pequeno número de bytes, e o emissor enche-o imediatamente outra vez com outro segmento pequeno.

Corrigir Isto do Lado do Recetor

- Podemos mitigar esta situação do lado do recetor.

- Depois de anunciar uma janela de tamanho zero, não anunciar uma janela aberta (ou seja, uma janela de receção maior que zero) até haver espaço para um maximum segment size (MSS) completo, ou pelo menos metade do buffer de receção estar livre.
  - Isto impede que o recetor seja alguma vez quem permite que segmentos pequenos (dominados por overhead) lhe sejam enviados no fluxo TCP.

Mas o Emissor Ainda Pode Portar-se Mal

- Mesmo um recetor bem comportado não resolve isto por completo — uma aplicação que escreve dados um byte de cada vez (pensem numa sessão de terminal interativa como o ssh ou o telnet mais antigo) ainda pode levar o emissor a enviar segmentos minúsculos por iniciativa própria (porque pede explicitamente ao TCP para esvaziar a ligação imediatamente).

O Algoritmo de Nagle

- **A regra:** se tanto os dados em buffer como a Janela de receção forem de pelo menos um segmento completo (MSS), enviar já um segmento completo. Caso contrário, se já houver dados por confirmar em trânsito, colocar os dados novos em buffer até chegar o próximo ACK; se nada estiver em trânsito, enviar de imediato o que houver.

- **A parte elegante:** o algoritmo é self-clocking — o próximo ACK acaba sempre por chegar (mesmo que por via de uma retransmissão despoletada por um timeout no emissor), por isso o TCP nunca precisa de um temporizador separado só para esta decisão.

Quando o Nagle Atrapalha

- **Para a maioria das aplicações, isto é invisível e dá desempenho adicional sem custo adicional.** Mas uma aplicação interativa que envia “uma tecla” de cada vez pode sentir a latência extra de esperar por um ACK antes de o byte seguinte sair.

- **Como evitar isto:** a opção de socket TCP_NODELAY desliga o algoritmo de Nagle para uma ligação que realmente precisa que cada byte seja enviado imediatamente (para que não haja buffering do lado do emissor).

Um Trade-Off, Não um Bug

Conclusão:
A Silly Window Syndrome e a sua correção do lado do emissor, o algoritmo de Nagle, vêm ambas da mesma tensão subjacente — enviar imediatamente para baixa latência, ou esperar e agrupar (agregando várias escritas da aplicação num único segmento de rede) para eficiência. As definições por omissão do TCP favorecem a eficiência, e o TCP_NODELAY é a opção da ligação TCP que permite escolher a outra opção.

Mais Uma Questão em Aberto

- Já sabemos como o TCP decide o que colocar num segmento, e quando o enviar. Uma questão da última aula continua em aberto: como é que o TCP sabe que um segmento se perdeu mesmo?

Um Timeout Não Serve Para Todos

- **Uma ligação ponto-a-ponto (física) tem um RTT fixo.** Uma ligação TCP não tem esse luxo: uma ligação pode ver um RTT de 1ms numa rede local, outra 300ms a atravessar o globo — e até o próprio RTT de uma única ligação pode variar por um fator de 5 ou mais ao longo da sua vida, à medida que as rotas e as condições de tráfego mudam.
- **Um timeout demasiado curto** retransmite dados que nunca se perderam — desperdiçando largura de banda;
- **Um timeout demasiado longo** faz o TCP demorar demasiado tempo a reagir a uma perda real, o que também deixa a ligação inativa durante um longo período de tempo.

A Resposta do TCP

- Medir o RTT continuamente, enquanto a ligação está a decorrer, e adaptar o valor do timeout ao que está realmente a ser observado.

O Algoritmo Original

- Sempre que o TCP envia um segmento, regista a hora do envio; quando o ACK chega, a diferença de tempo entre enviar o segmento e receber o ACK torna-se um SampleRTT (*amostra de RTT*).

- EstimatedRTT = α × EstimatedRTT + (1 − α) × SampleRTT — uma média móvel ponderada (α entre 0.8 e 0.9, favorecendo o histórico suavizado em detrimento de qualquer amostra isolada, porque, como sabemos, a rede pode apresentar variações isoladas de latência).

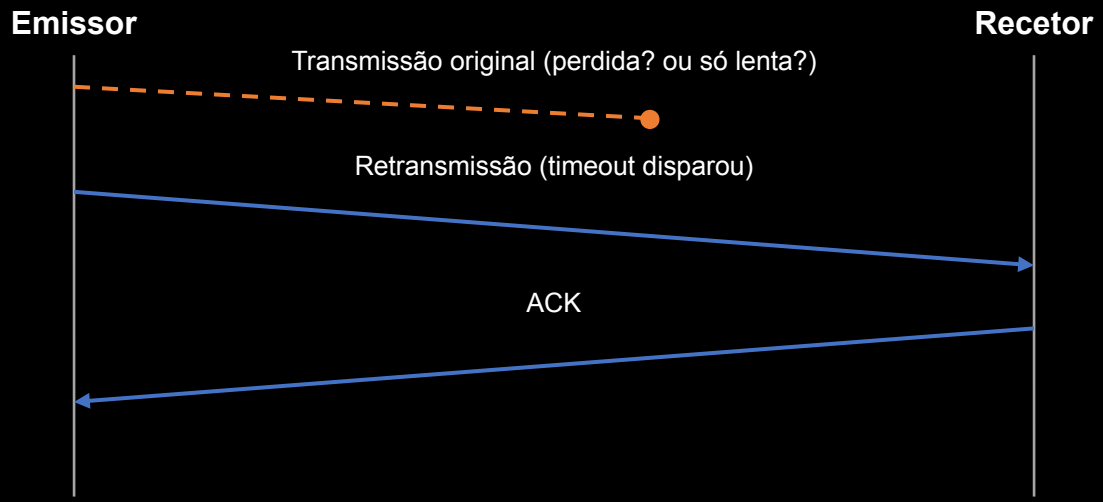
Definir o Timeout

- TimeOut = 2 × EstimatedRTT.
  - Duplicar dá margem real de segurança — um único RTT ligeiramente mais lento não deve despoletar uma retransmissão imediata, o que permite evitar gastar largura de banda desnecessária na rede.

Uma Falha Subtil

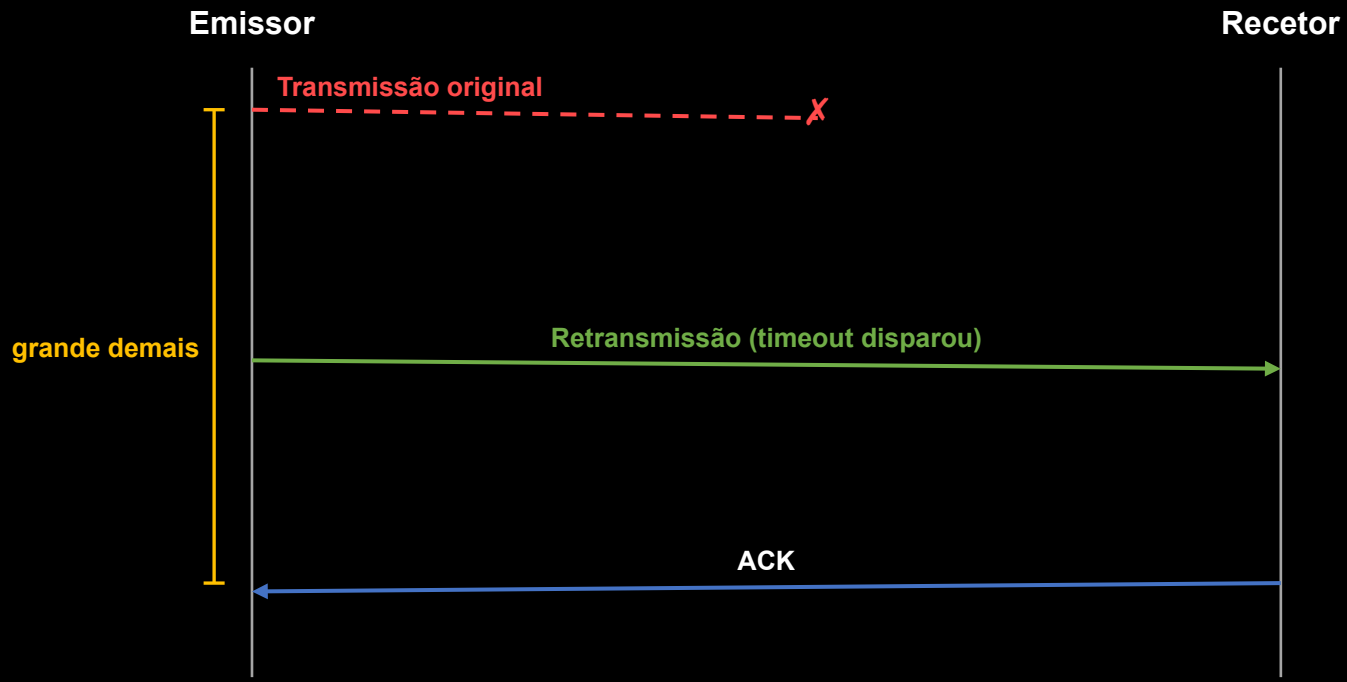
- Isto funcionou durante anos, até alguém reparar numa ambiguidade real escondida dentro do algoritmo: o que acontece quando um segmento tem de ser retransmitido?

A Ambiguidade de Karn/Partridge



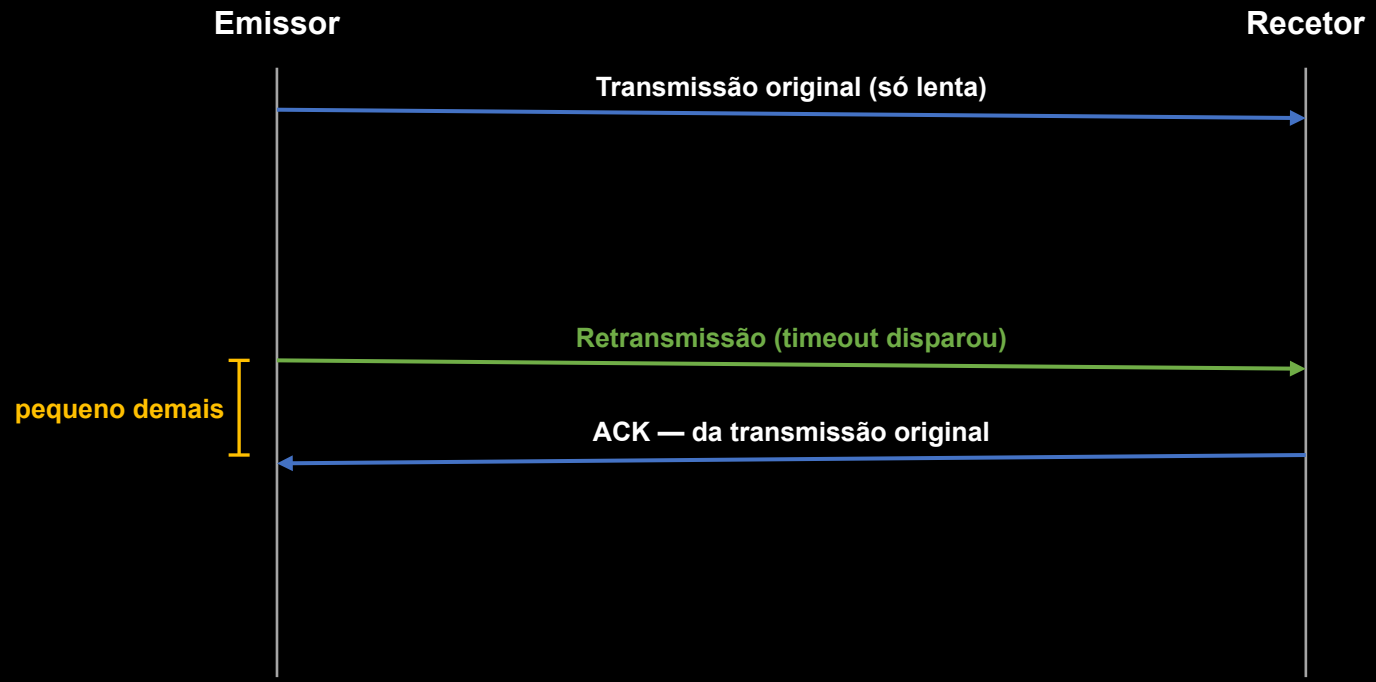
Este ACK pertence à transmissão original, ou à retransmissão? Adivinhar mal faz o SampleRTT ficar extremamente grande ou pequeno demais.

A Ambiguidade de Karn/Partridge



(a) A original perdeu-se: o ACK responde à retransmissão. Medir a partir do envio original torna o SampleRTT demasiado grande.

A Ambiguidade de Karn/Partridge



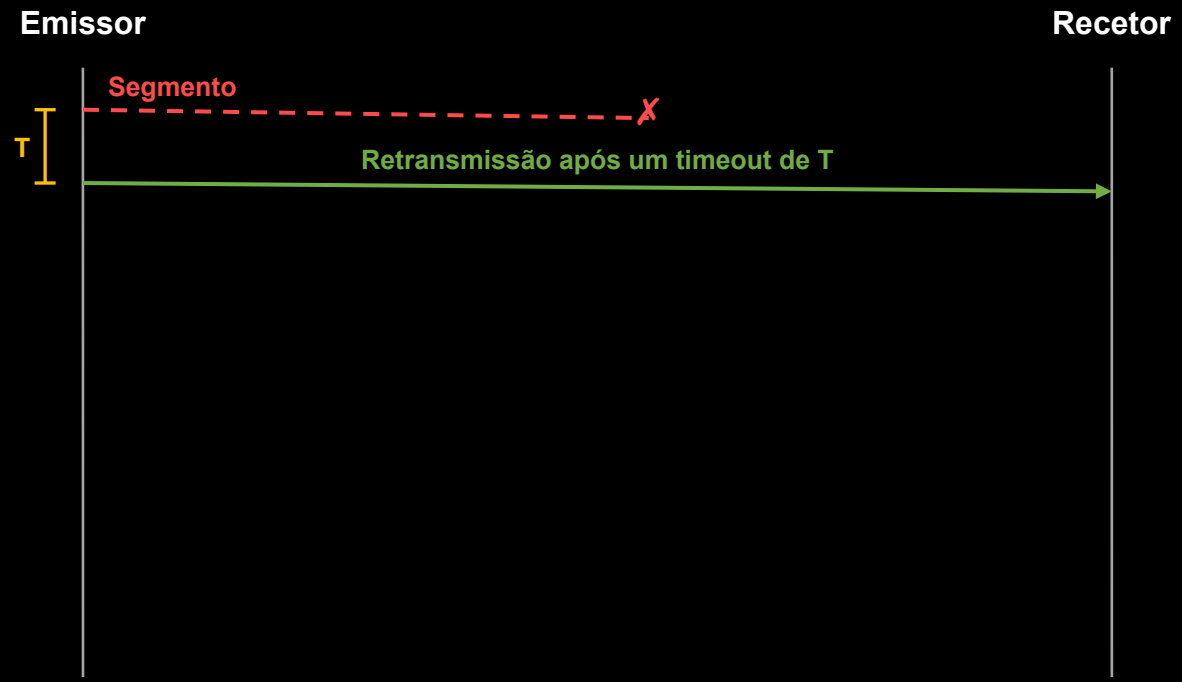
(b) A original era só lenta: o seu ACK chega logo depois da retransmissão. Medir a partir da retransmissão torna o SampleRTT demasiado pequeno.

A Correção de Karn/Partridge

- **O problema:** um ACK não diz que transmissão (original ou repetição) está a confirmar — se um segmento foi retransmitido, adivinhar mal a qual delas corresponde o ACK faz o SampleRTT ficar completamente errado, possivelmente em qualquer direção.

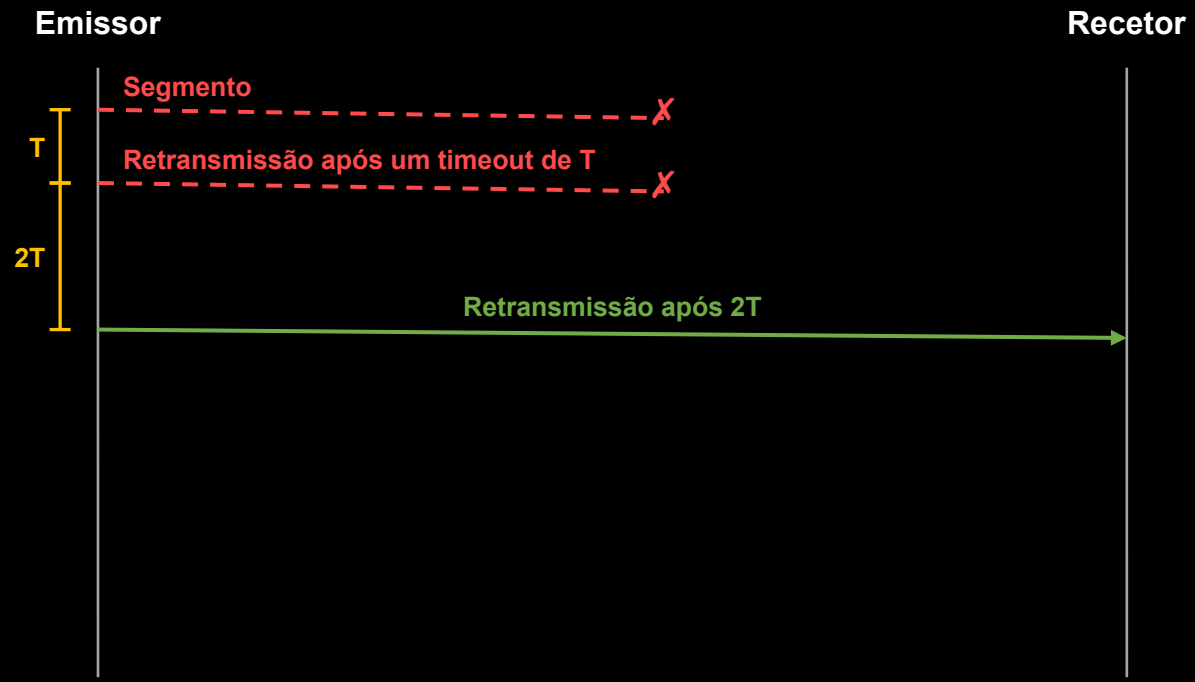
- **A correção:** nunca amostrar o RTT num segmento que foi retransmitido — só considerar SampleRTT de segmentos que foram enviados exatamente uma vez.
- **Além disso:** numa retransmissão, duplicar o timeout em vez de o recalcular a partir do EstimatedRTT — a isto chama-se **exponential backoff**.

Exponential Backoff



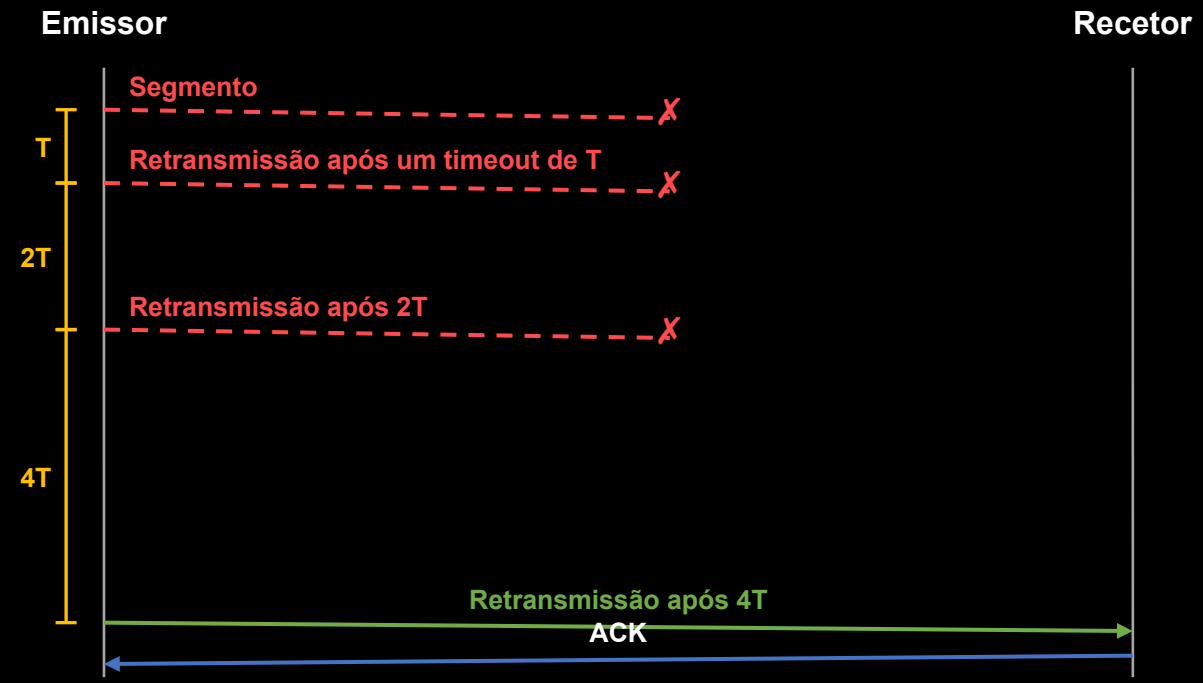
O segmento perde-se; quando o temporizador (T) dispara, o TCP retransmite…

Exponential Backoff



…esta também se perde, por isso o TCP espera o dobro do tempo (2T) antes de tentar de novo…

Exponential Backoff



…e duplica outra vez (4T) até um ACK finalmente chegar. Uma rede em dificuldades ganha mais espaço para respirar a cada falha.

Ainda Não É Toda a História

- A correção de Karn e Partridge resolveu a ambiguidade, mas os timeouts do TCP ainda não estavam totalmente certos — o algoritmo original tinha mais um ponto cego.

Jacobson/Karels: Ter em Conta a Variância

- O algoritmo original ignora o quanto o SampleRTT varia — uma ligação instável merece um timeout mais cauteloso do que uma estável, mesmo que a média de RTT observada seja a mesma.

- Difference = SampleRTT − EstimatedRTT
- EstimatedRTT += δ × Difference
- Deviation += δ × (|Difference| − Deviation) — com δ = 1/8 na prática

A Fórmula Real

- TimeOut = μ × EstimatedRTT + φ × Deviation (μ=1, φ=4 na prática)

- Um Deviation grande faz o timeout ajustar-se a essa variância, não só à média — exatamente a lacuna que o algoritmo original tinha.

O que permite voltar a valores inferidos para o Timeout?

- **A regra de Karn/Partridge desativa efetivamente a amostragem de RTT durante um período com retransmissões,** enquanto o **exponential backoff** continua a duplicar o timeout — então o que é que traz o timeout de volta ao normal? (O EstimatedRTT nunca subiu: simplesmente deixou de ser atualizado.)

- **A resposta:** o segmento seguinte que é confirmado sem precisar de retransmissão produz um SampleRTT novo e fiável — esse único RTT limpo é o que permite ao TCP largar o timeout de backoff inflacionado e voltar a usar um timeout com base na estratégia adaptativa normal.

Um Tipo Diferente de Silêncio

- **A retransmissão adaptativa trata de um emissor à espera de um ACK para dados que já enviou.** Mas e uma ligação que está simplesmente inativa — ninguém a enviar nada, de nenhum dos lados?
- **O temporizador de retransmissão só dispara quando há algo por confirmar em trânsito.** Uma ligação inativa não tem nada em aberto, por isso o temporizador nunca dispara — por mais tempo que o outro lado já tenha desaparecido.
- **Se o outro lado crashou, reiniciou, ou a rede se particionou silenciosamente,** o próprio estado de uma ligação inativa pode sobreviver indefinidamente — nada está a sondar o outro endpoint para identificar que ocorreu uma falha.

Keep-Alive: Sondar o Silêncio

- **Um mecanismo de keep-alive envia periodicamente uma sonda numa ligação que, de outro modo, estaria inativa,** só para provocar uma resposta.

- **Uma resposta significa que o outro lado — e o caminho até ele — ainda lá está:** repor o temporizador de inatividade e não dizer mais nada. Sem resposta ao fim de tentativas suficientes significa que a ligação está mesmo morta, e o lado local pode finalmente desistir dela.
  - **Nem todas as ligações querem isto:** uma sonda numa falha breve da rede pode matar uma ligação saudável — daí vir desligado por omissão (SO_KEEPALIVE) e, ligado, só emite sondas após 2 h de silêncio (RFC 1122).

Os Timeouts Também São Sinais

Conclusão:

um timeout não é só “retransmitir” — é também a primeira pista do TCP de que a própria rede pode estar com dificuldades, exatamente Aquilo que nos falta discutir: o controlo de saturação.

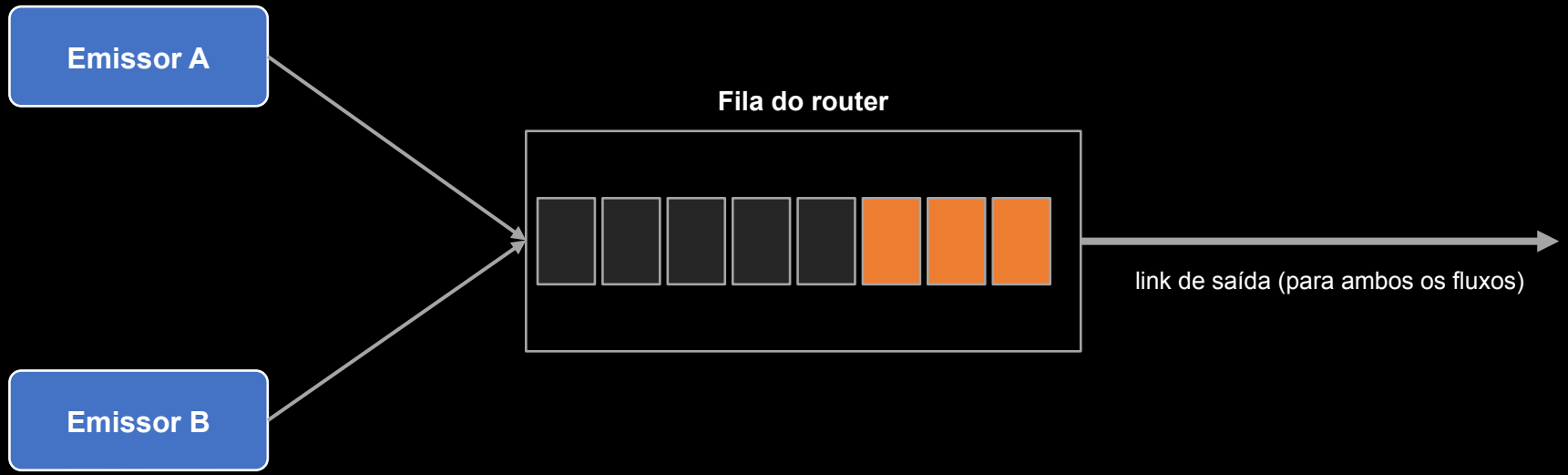
Uma Coisa Por Resolver

- O controlo de fluxo impede um emissor rápido de atropelar um recetor lento. Mas o que impede um emissor rápido de sobrecarregar a própria rede — os links e routers/switches pelo meio, que não têm qualquer interferência nos valores propagados na AdvertisedWindow?
- A última coisa que nos falta ver no TCP: os mecanismos de Controlo de Saturação do TCP.

Colapso por Saturação

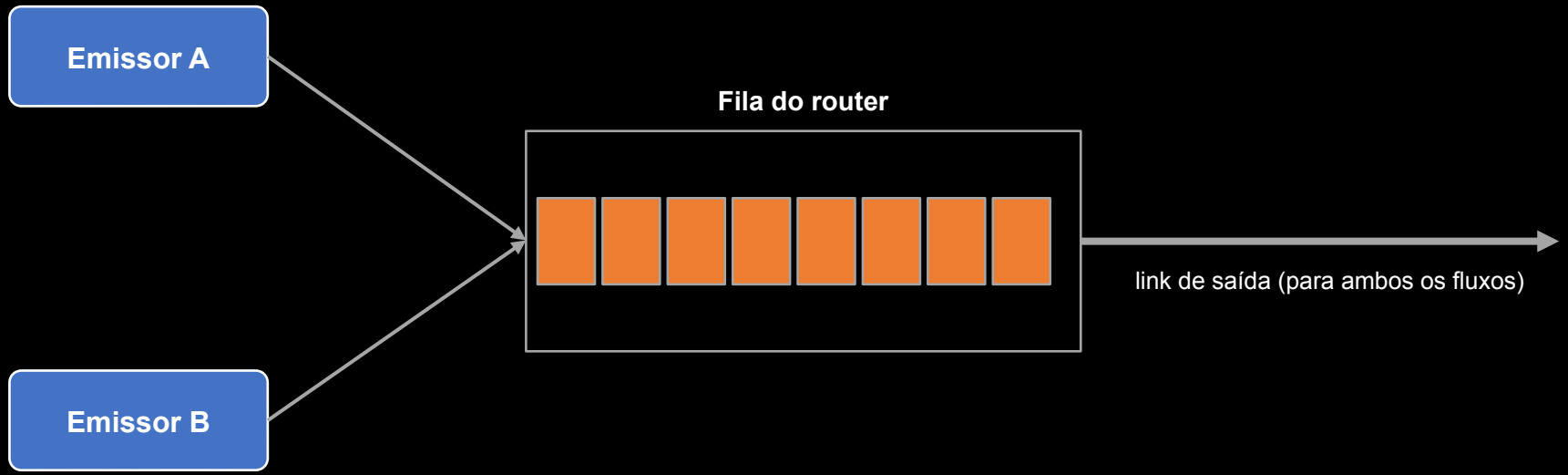
- **No final dos anos 1980, a Internet quase parou de funcionar:** os hosts enviavam tão depressa quanto o AdvertisedWindow permitia — só o recetor era consultado.
- **As filas dos routers transbordavam e descartavam pacotes;** os emissores davam timeout e retransmitiam, acrescentando ainda mais tráfego a uma rede já sobrecarregada.
- **O throughput útil entrava em colapso** enquanto os links se mantinham ocupados a transportar cópias dos mesmos dados.
- **A correção de Van Jacobson (com Michael Karels, 1988):** cada emissor TCP estima quanto é que a rede aguenta — introduzido o mecanismos de controlo de saturação.

Quando Todos Enviam Demasiado



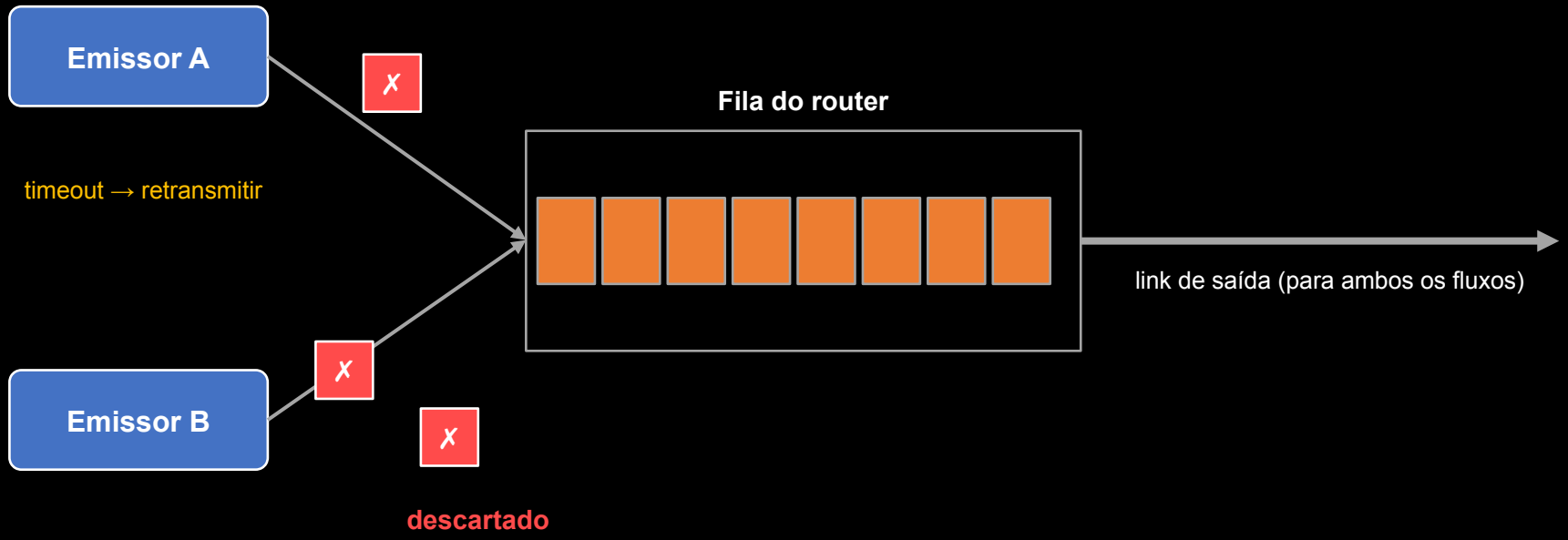
Abaixo da capacidade: a fila do router absorve rajadas curtas.

Quando Todos Enviam Demasiado



Os emissores empurram mais forte: a fila enche-se.

Quando Todos Enviam Demasiado



A fila transborda: os pacotes são descartados, os emissores dão timeout e retransmitem — mais carga, não menos.

A Estratégia do TCP

- **Sem reservas:** o TCP simplesmente envia pacotes para a rede e observa o que acontece.

- **Reagir ao que observa:** abrandar quando os pacotes se perdem, acelerar quando continuam a chegar ao destino.

- **Self-clocking:** cada ACK que regressa significa que um pacote saiu da rede (entregue ao recetor) — por isso é seguro colocar mais um pacote na rede.

Quem É o Limitador?

- O controlo de fluxo abranda o emissor. O controlo de saturação também abranda o emissor.
- Então qual é a diferença real entre os dois?

Dois Limitadores Diferentes

- **Controlo de fluxo:** o limitador é o recetor — o seu espaço livre em buffer, anunciado no AdvertisedWindow.

- **Controlo de saturação:** o limitador é a rede — os links e routers pelo meio, que nunca anunciam nada.

- **Um emissor tem de respeitar os dois,** sempre.

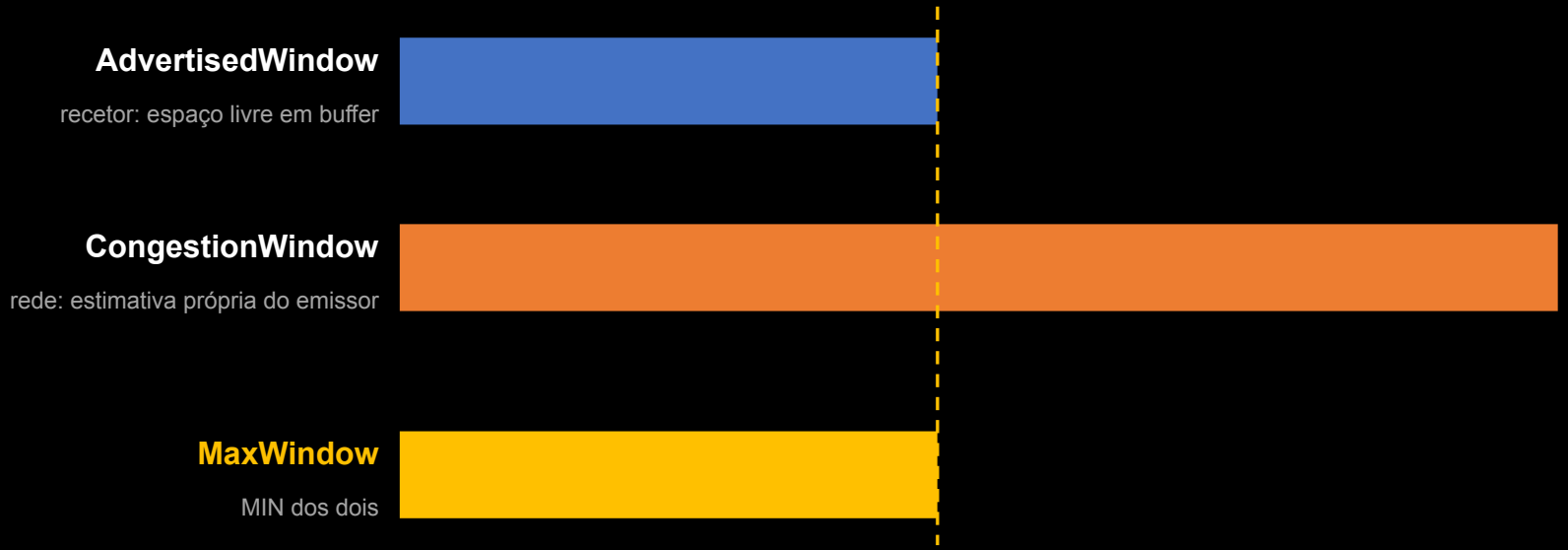
CongestionWindow

- **Uma nova variável por ligação:** CongestionWindow — quantos dados a rede consegue aguentar deste emissor neste momento, em segurança (sem saturar a rede e levar a perda de pacotes).

- **Ninguém envia esse valor:** ao contrário do AdvertisedWindow, ninguém diz ao emissor o seu valor — o emissor tem de o inferir.

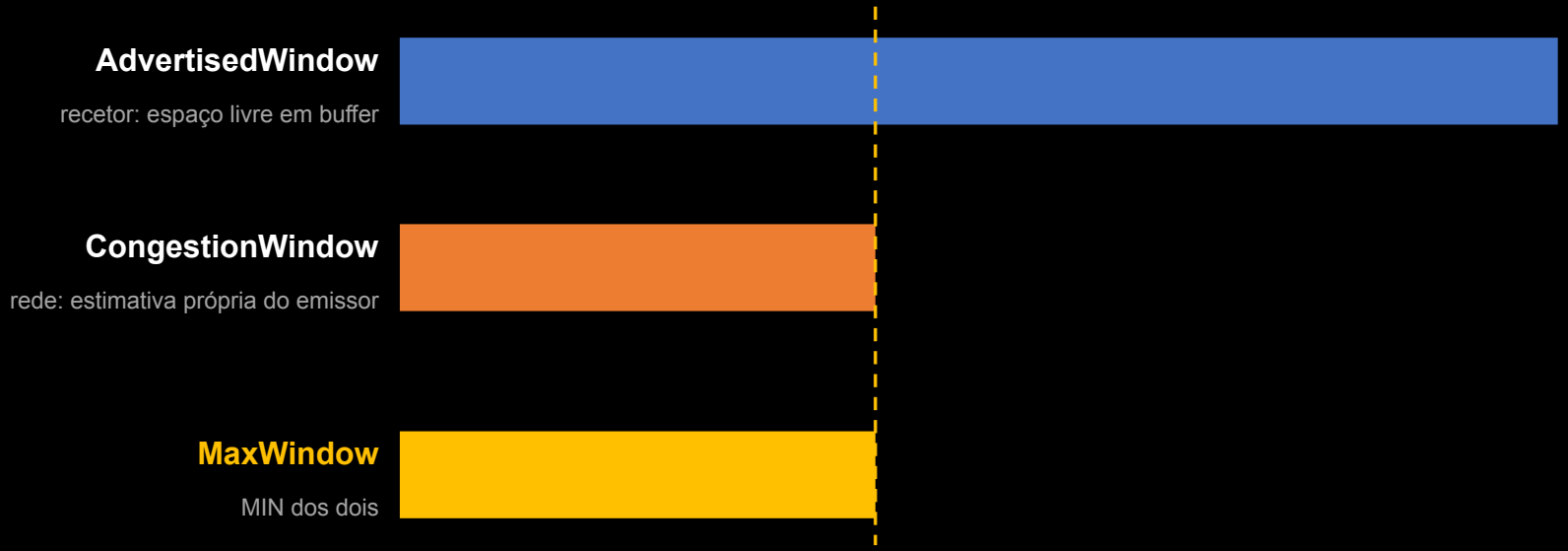
- **MaxWindow = MIN(CongestionWindow, AdvertisedWindow)**
- **EffectiveWindow = MaxWindow − (LastByteSent − LastByteAcked)** — a fórmula da última aula, com um termo substituído.

O Menor é que Ganha



Um recetor lento: o AdvertisedWindow é o menor dos dois — o controlo de fluxo decide.

O Menor é que Ganha



Uma rede saturada: o CongestionWindow é o menor — o controlo de saturação decide.

Como É Que o Emissor Sabe?

- Ninguém diz ao emissor o quão saturada a rede está.
- Então o que é que ele consegue observar que lhe permita fazer ter uma estimativa?

A Perda Como Sinal

- **Um timeout significa que um pacote se perdeu —** e em links com fios isso é quase sempre porque um router o descartou, não por causa de um erro de transmissão.
- **Por isso o TCP interpreta um timeout como sinal de saturação da rede** e reduz o CongestionWindow.
- **Quando os pacotes continuam a chegar ao recetor,** aumenta o CongestionWindow para usar capacidade que possa ter ficado disponível na rede.
- **A regra para aumentar e reduzir** chama-se AIMD: Additive Increase, Multiplicative Decrease (aumento aditivo, diminuição multiplicativa).

Diminuição Multiplicativa

- **A cada perda, reduzir o CongestionWindow para metade:** 16 → 8 → 4 → 2 → 1 pacotes após perdas sucessivas (na realidade é mais complicado do que isto: depois de um timeout, essa metade fica guardada como limiar e a janela recomeça em 1).

- **Nunca abaixo de um pacote:** o CongestionWindow não pode cair abaixo de um Maximum Segment Size (MSS).

- **Rápido, de propósito:** uma janela demasiado grande continua a alimentar a saturação — sair rapidamente desse estado é o que mais importa para poder dar tempo ”à rede” para recuperar.

Aumento Aditivo

- **Em cada RTT em que uma janela inteira de pacotes é confirmada,** acrescentar um pacote ao CongestionWindow.

- **Na prática, um pouco a cada ACK:** com 10 pacotes em trânsito, cada ACK acrescenta um décimo de pacote — por isso um RTT acrescenta cerca de um.

- **Lento, de propósito:** a tentar cuidadosamente descobrir qual a capacidade de rede disponível, um pacote de cada vez.

Aumento Aditivo, RTT a RTT

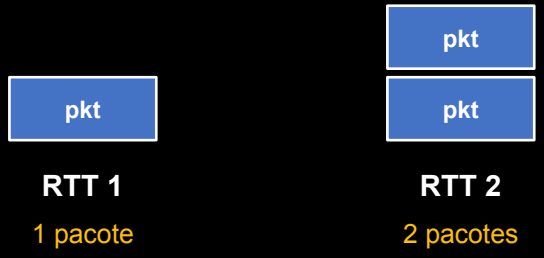


RTT 1

1 pacote

Mais um pacote em trânsito a cada RTT: crescimento linear.

Aumento Aditivo, RTT a RTT



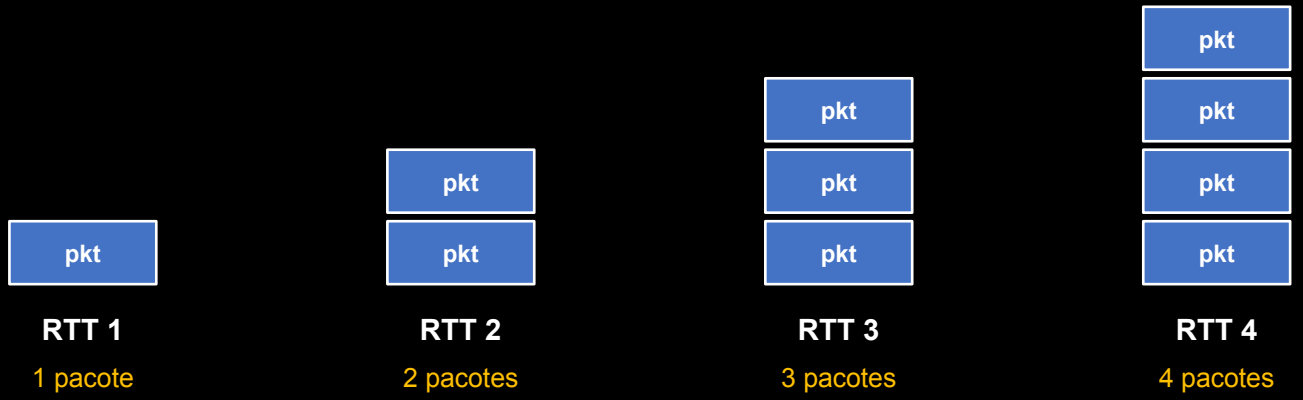
Mais um pacote em trânsito a cada RTT: crescimento linear.

Aumento Aditivo, RTT a RTT



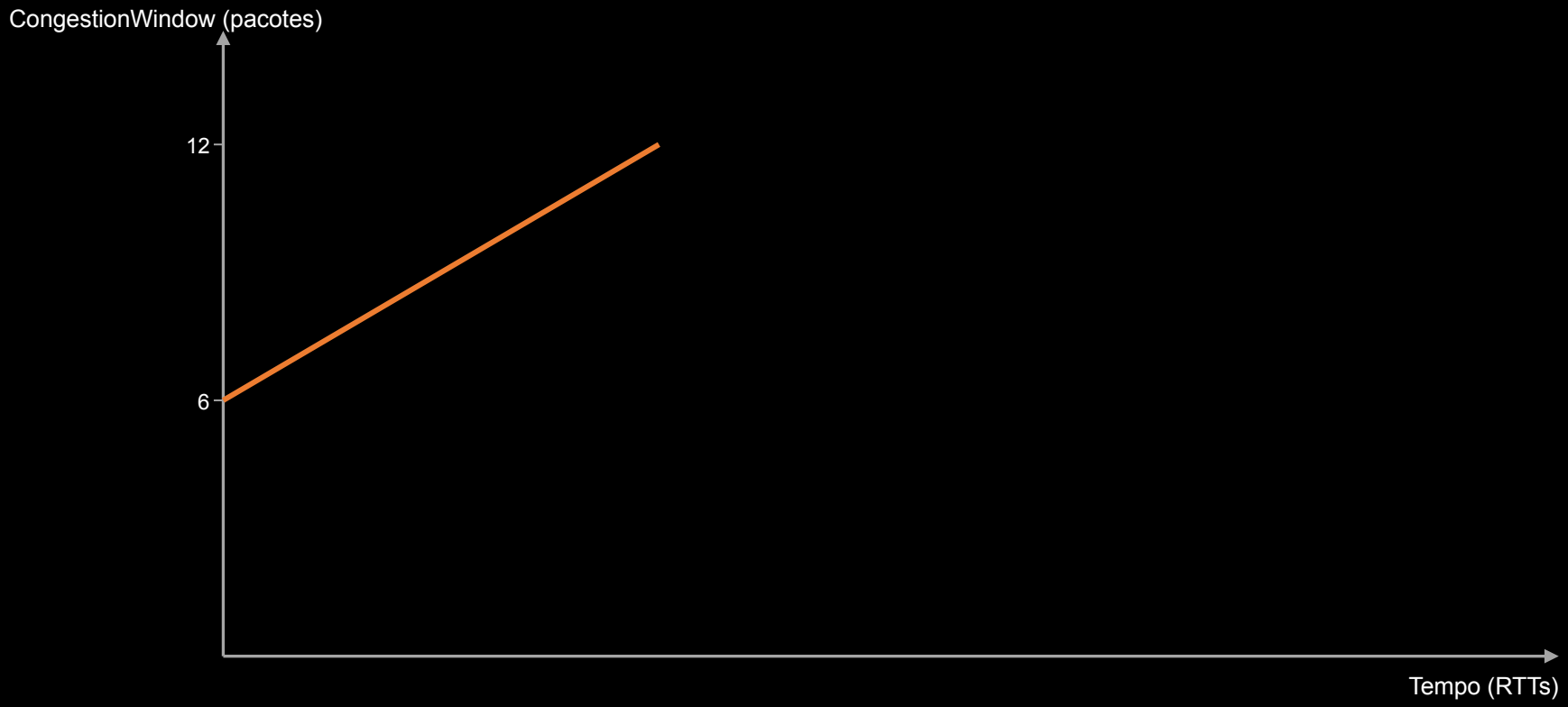
Mais um pacote em trânsito a cada RTT: crescimento linear.

Aumento Aditivo, RTT a RTT



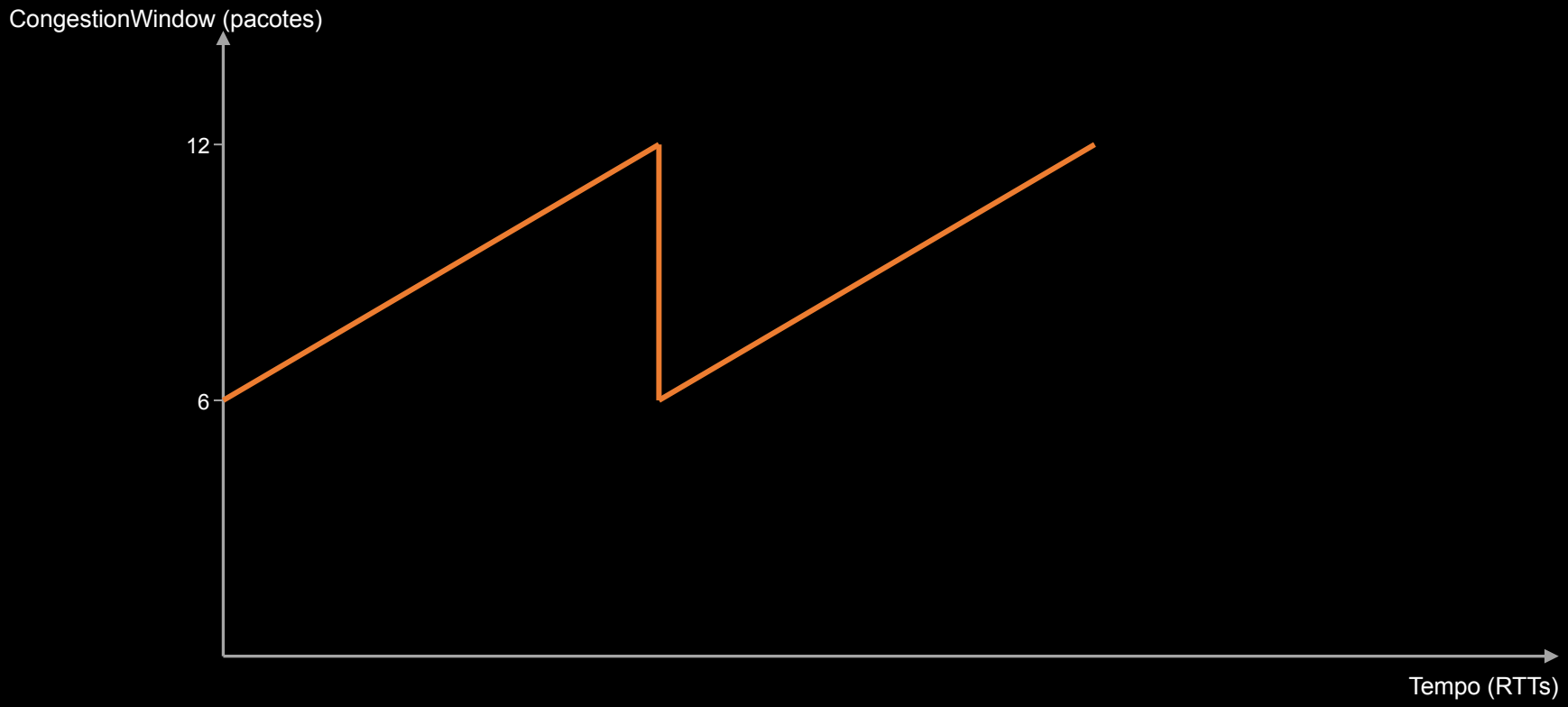
Mais um pacote em trânsito a cada RTT: crescimento linear.

O Dente de Serra



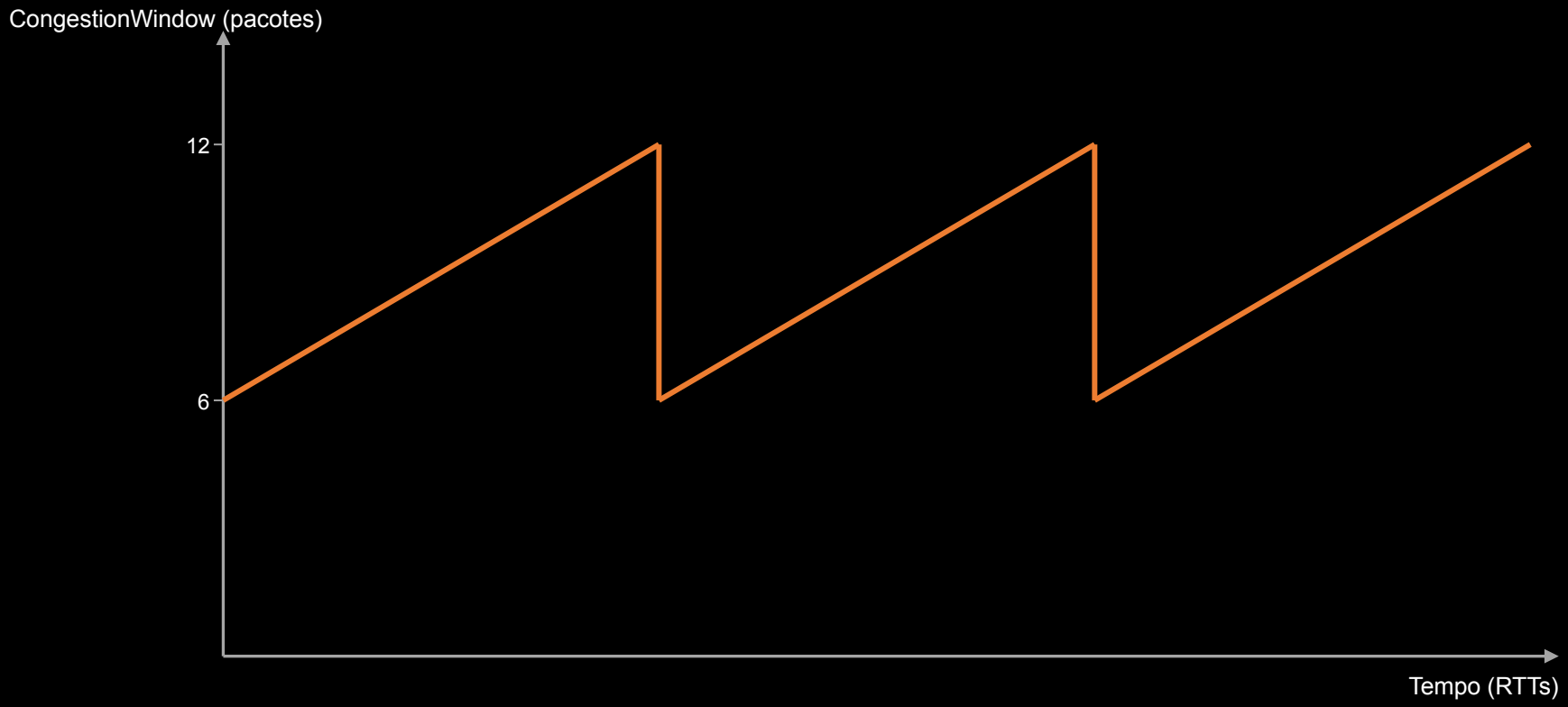
Crescimento linear, um pacote por RTT, até haver uma perda…

O Dente de Serra



…reduzir para metade, depois crescer linearmente outra vez…

O Dente de Serra



…durante toda a vida da ligação: o dente de serra do TCP.

Porquê Assimétrico?

Conclusão:
uma janela demasiado grande piora a saturação (cada perda despoleta uma retransmissão), enquanto uma demasiado pequena só desperdiça um pouco de capacidade. Por isso o TCP recua rapidamente e volta a tentar aumentar o uso da rede devagar — e esta assimetria é o que mantém todo o sistema (com vários fluxos TCP simultâneos) estável.

A Começar do Zero

- O aumento aditivo acrescenta um pacote por RTT.
- Uma ligação nova num caminho rápido pode conseguir manter milhares de pacotes em trânsito — quanto tempo demorariamos a lá chegar aumentando apenas um pacote de cada vez?

Slow Start

- **Começar com CongestionWindow = 1 pacote,** e acrescentar um pacote por cada ACK que chega.

- **Cada pacote enviado produz um ACK,** por isso a janela duplica a cada RTT: 1, 2, 4, 8, 16…

- **Exponencial, não linear:** uma ligação atinge uma janela grande em meia dúzia de RTTs em vez de milhares.

Slow Start, RTT a RTT

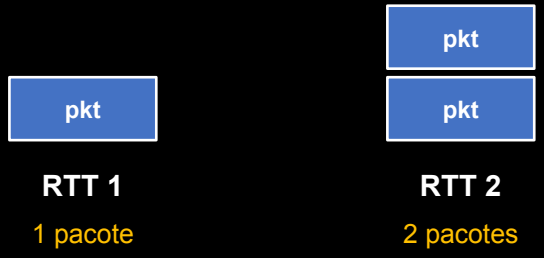


RTT 1

1 pacote

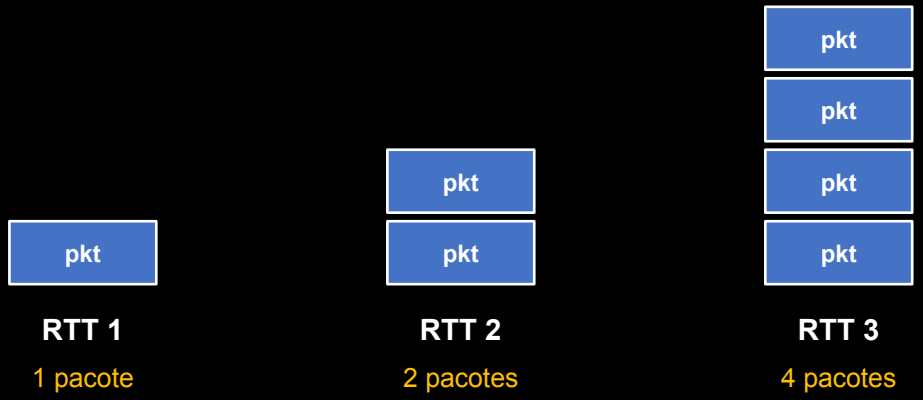
A duplicar a cada RTT: crescimento exponencial.

Slow Start, RTT a RTT



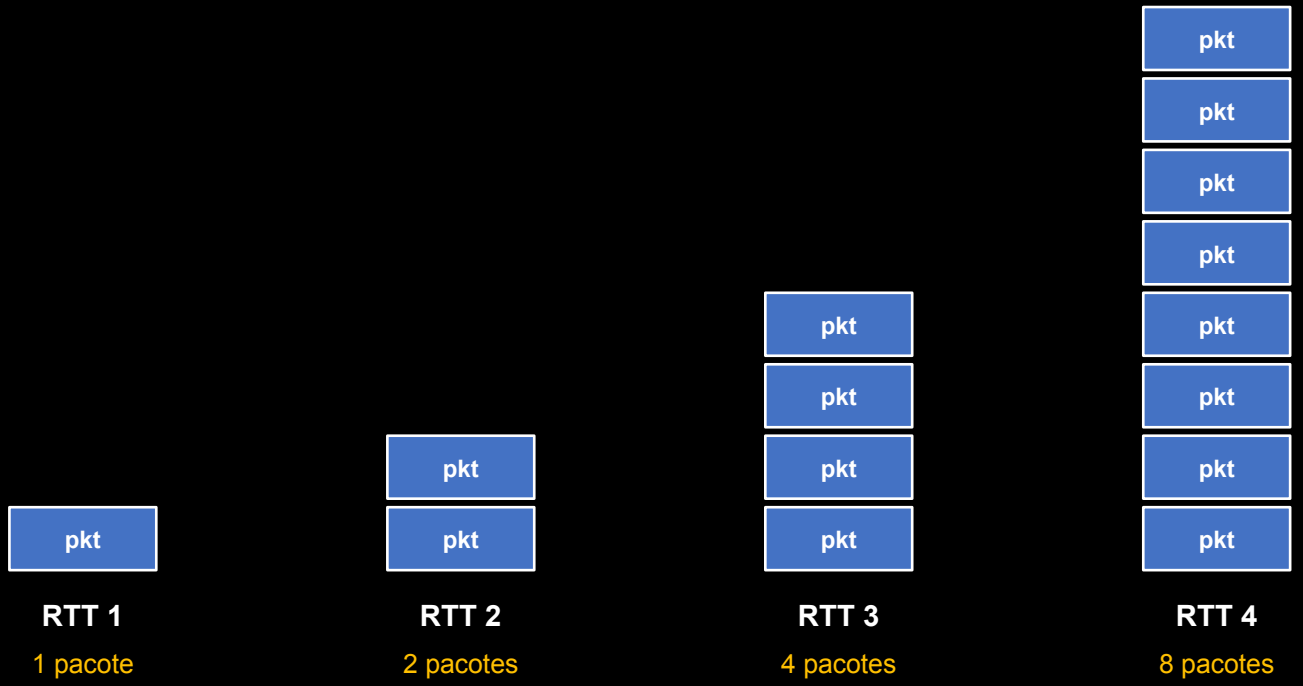
A duplicar a cada RTT: crescimento exponencial.

Slow Start, RTT a RTT



A duplicar a cada RTT: crescimento exponencial.

Slow Start, RTT a RTT



A duplicar a cada RTT: crescimento exponencial.

Porquê “Slow”?

- O slow start cresce exponencialmente — muito mais depressa do que o aumento aditivo.
- Então porque é que se chama slow (lento)?

Lento Comparado com Quê?

- **Antes de 1988, o TCP começava por enviar um AdvertisedWindow inteiro de uma vez —** uma rajada que as filas dos routers não conseguiam absorver e que só aumentava a congestão de rede no caso dela existir.

- **O slow start distribui a carga introduzida por vários RTTs,** deixando que os ACKs que regressam marquem o ritmo de saída dos pacotes.

- **“Lento” comparado com essa possível rajada de pacotes iniciais —** não comparado com o aumento aditivo.

Quando É Que o Slow Start Corre?

- **No início de uma ligação:** o emissor não faz ideia se o caminho até ao destino tem uma capacidade de 1 Mbps ou de 40 Gbps.

- **Depois de um timeout:** o TimeOut está definido bem acima de um RTT normal, por isso, quando dispara, quaisquer pacotes sobreviventes já teriam sido confirmados — não resta nada em trânsito para marcar o ritmo de novos envios, e o emissor volta a subir o ritmo a partir de um único pacote.

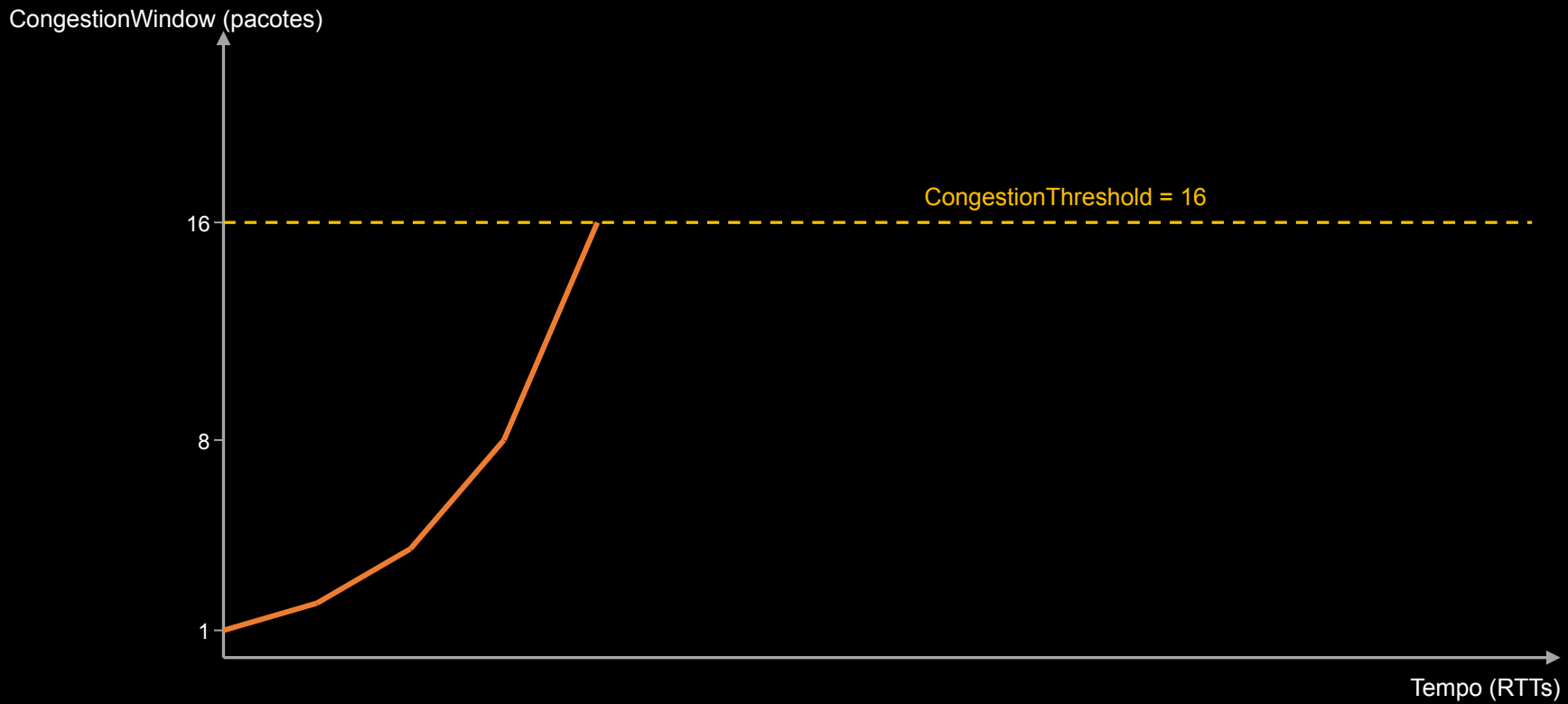
O Limiar de Saturação

- **Num timeout, o TCP guarda metade do CongestionWindow atual** numa segunda variável, o CongestionThreshold.

- **O CongestionWindow recomeça em um pacote,** e o slow start fá-lo crescer exponencialmente…

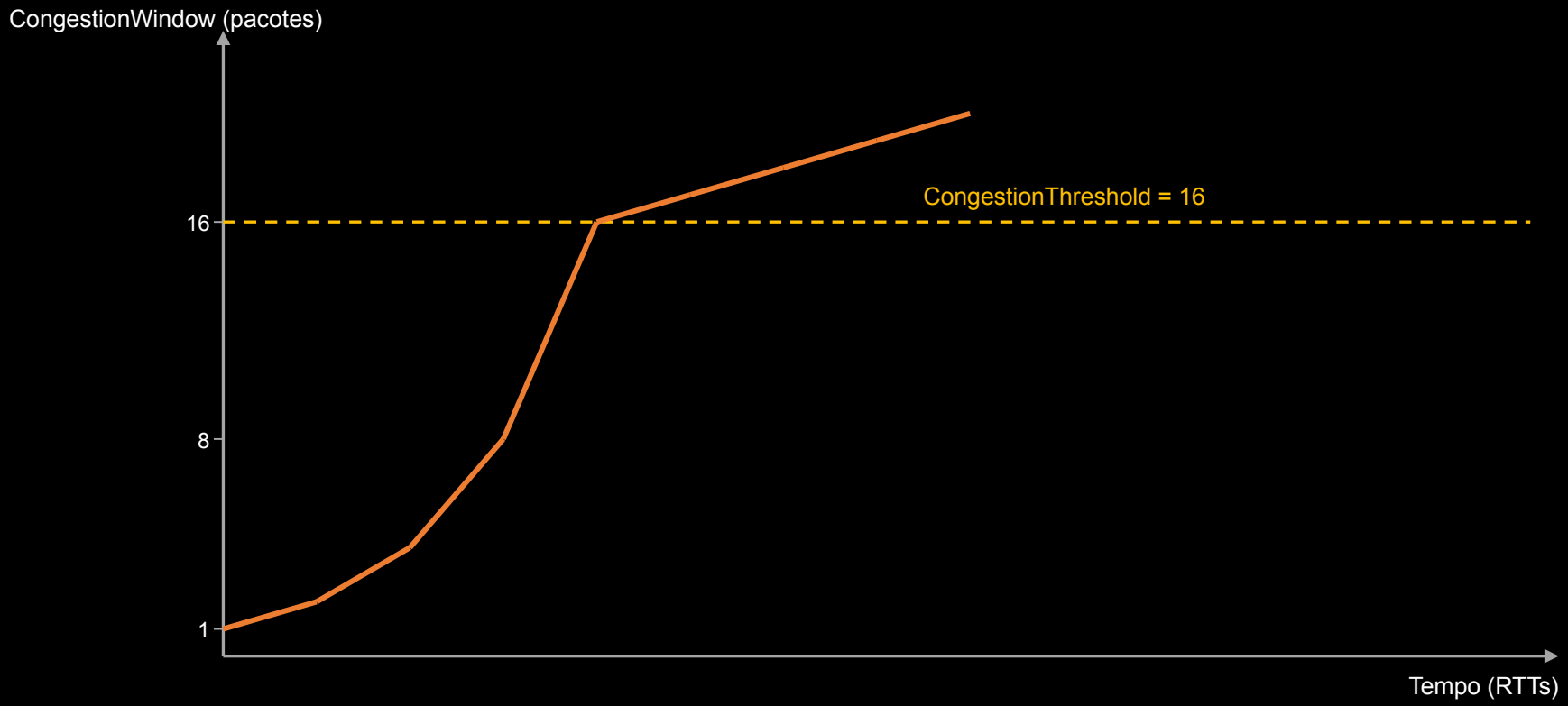
- **…até atingir o CongestionThreshold,** onde o TCP passa para aumento aditivo — a metade do nível onde os problemas começaram da última vez.

O Slow Start Encontra o Limiar



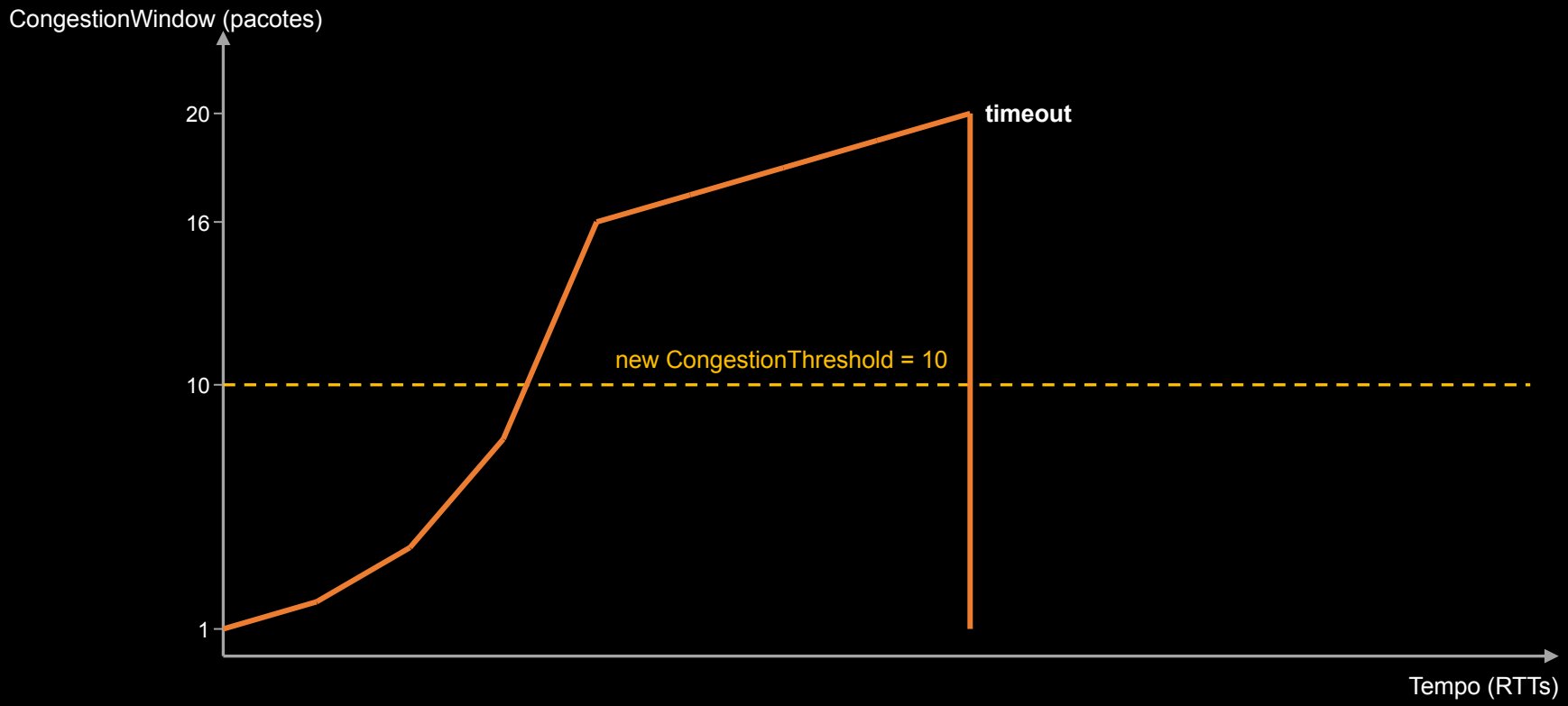
Slow start: a duplicar a cada RTT, até um limiar de 16 deixado por uma perda anterior (uma ligação acabada de abrir ainda não tem limiar).

O Slow Start Encontra o Limiar



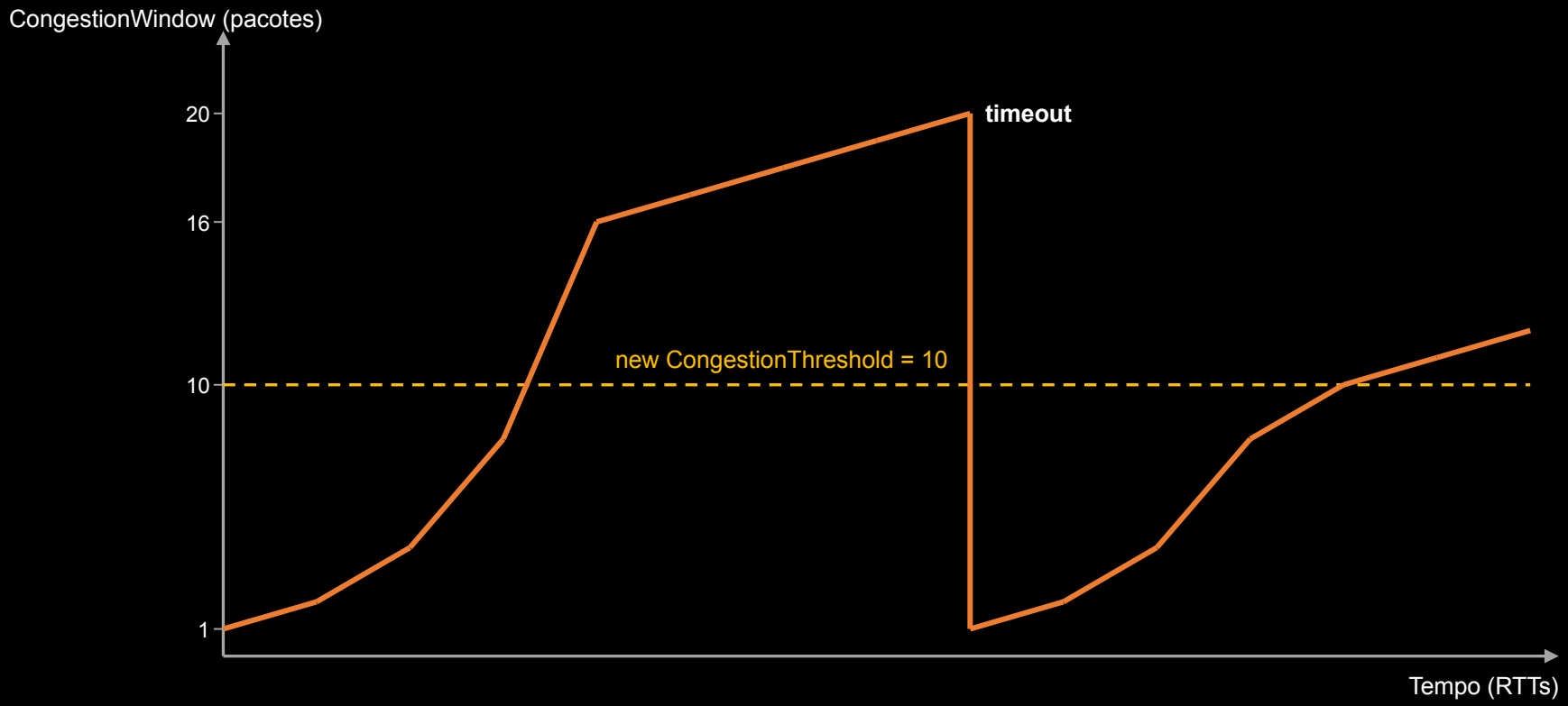
Acima do limiar: aumento aditivo, um pacote por RTT.

O Slow Start Encontra o Limiar



Timeout aos 20: o limiar passa a 10 e a janela recomeça em 1.

O Slow Start Encontra o Limiar



Slow start até ao novo limiar, depois crescimento linear outra vez.

O Preço de Duplicar

- **O slow start é mais agressivo mesmo no fim:** depois de 16 pacotes passarem, a janela salta para 32.

- **Se o caminho só aguentar 16,** até 16 desses 32 pacotes podem perder-se de uma vez.

- **Quanto mais rápida a rede, maior a perda possível —** o preço de descobrir a capacidade depressa.

Um Timeout Faz Duas Coisas

- **Retransmissão:** reenviar o segmento em falta — com o temporizador a duplicar a cada nova tentativa (exponential backoff, que discutimos hoje).

- **Resposta de saturação:** CongestionThreshold = CongestionWindow / 2; CongestionWindow = 1; slow start outra vez.

- **Ambas endereçam a mesma situação:** a rede está com dificuldades — recuar com força, e só depois voltar lentamente a subir o ritmo de transmissão.

Temos de Esperar pelo Temporizador?

- Os timeouts são grosseiros: um emissor pode ficar parado muito tempo antes de o temporizador disparar.
- Haverá alguma indicação que o emissor possa observar mais cedo, que lhe possa indicar que um pacote se perdeu? Já discutimos isto antes, no contexto da janela deslizante.

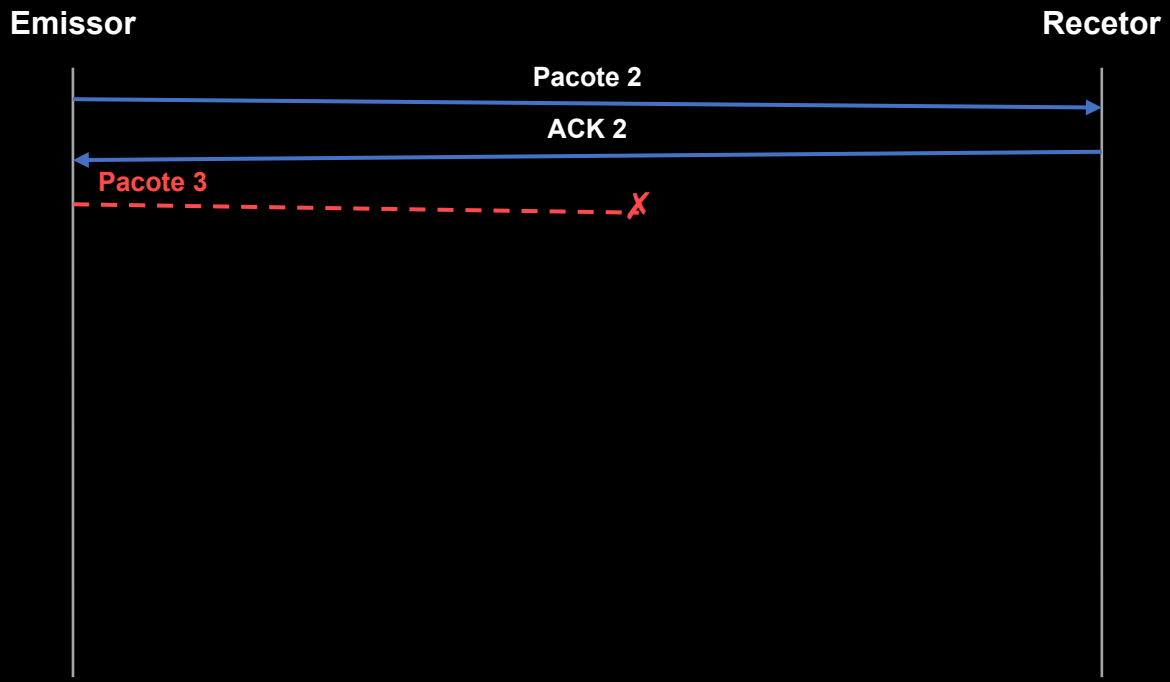
ACKs Duplicados

- **O recetor confirma cada segmento que recebe,** mesmo um que chegue fora de ordem — repetindo o seu último ACK relativo aos últimos bytes que recebeu em ordem.

- **Um ACK repetido é um ACK duplicado:** um sinal, não uma prova, de que algo se perdeu na rede (mas não tudo) — exatamente o que observámos com os ACKs cumulativos há duas aulas.

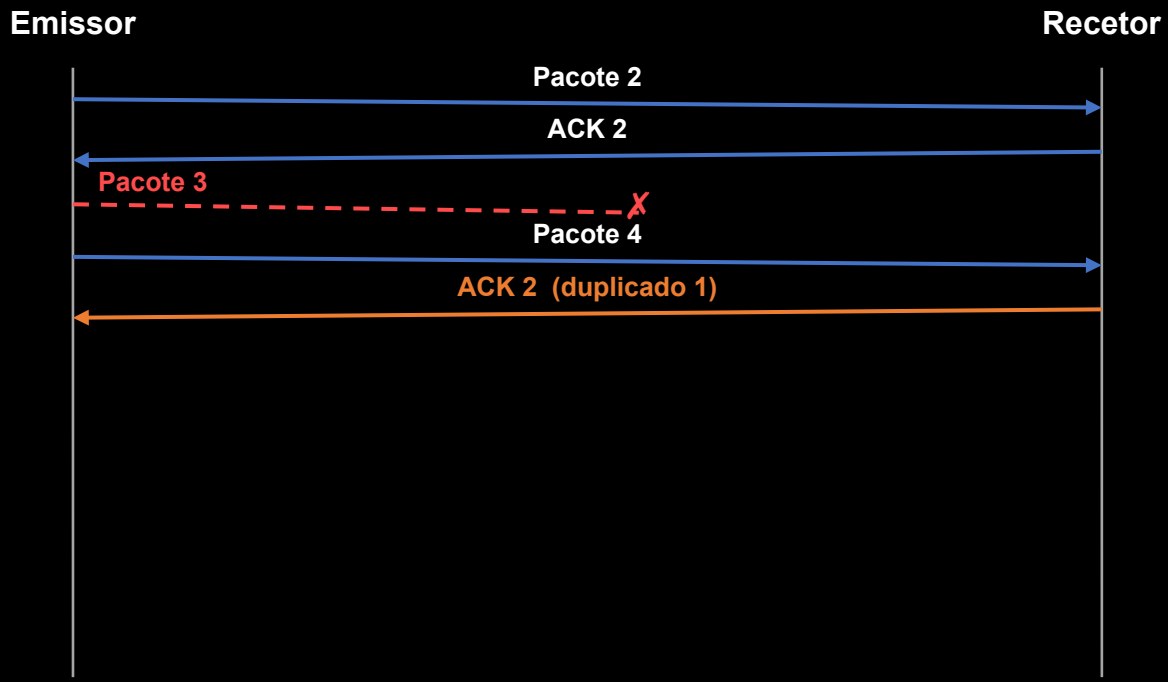
- **O TCP espera por três duplicados** antes de agir, para que um pacote que foi só atrasado ou reordenado não despolete uma retransmissão que não é necessária.

Fast Retransmit



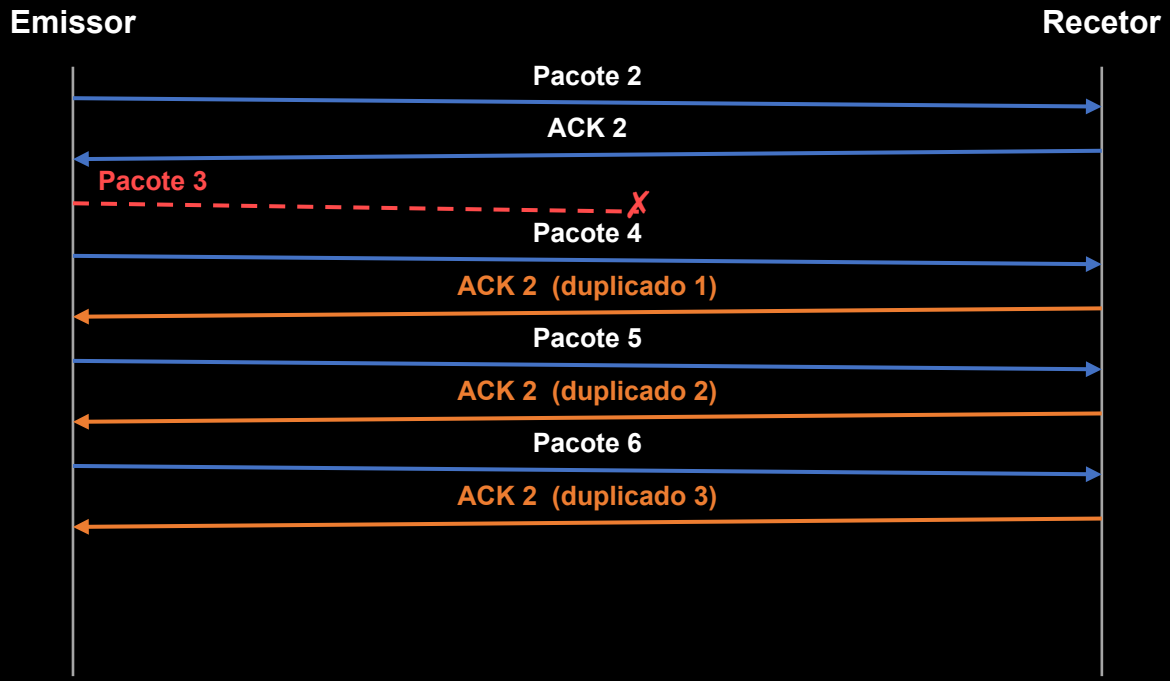
O Pacote 3 perde-se pelo caminho.

Fast Retransmit



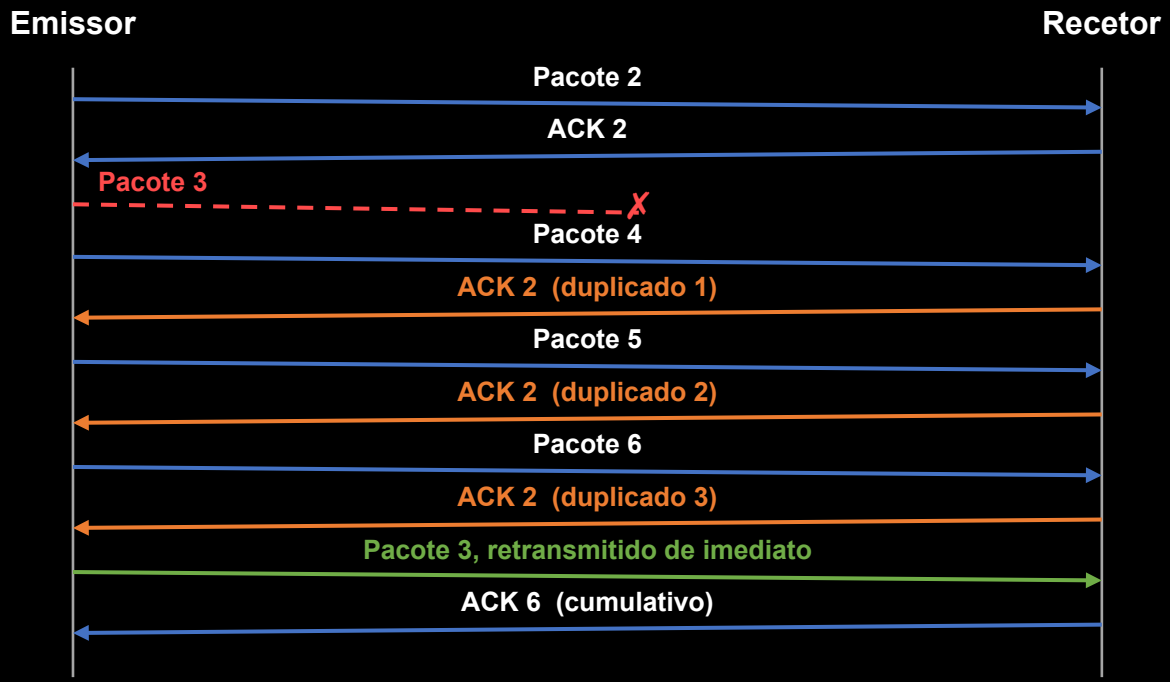
O Pacote 4 chega fora de ordem: o recetor repete o ACK 2. (Convenção do livro: ACK n = "tudo até ao pacote n"; na rede seria o ACK 3.)

Fast Retransmit



Três ACKs duplicados para 2: o emissor deixa de esperar pelo temporizador…

Fast Retransmit



…retransmite o pacote 3 de imediato, e um único ACK cumulativo cobre tudo até ao 6.

Fast Retransmit

- **Retransmitir ao terceiro ACK duplicado,** sem esperar pelo timeout.

- **Na prática, elimina cerca de metade dos timeouts longos,** para cerca de mais 20% de throughput.

- **Não substitui o temporizador:** com uma janela pequena pode não haver pacotes suficientes em trânsito para produzir três duplicados.

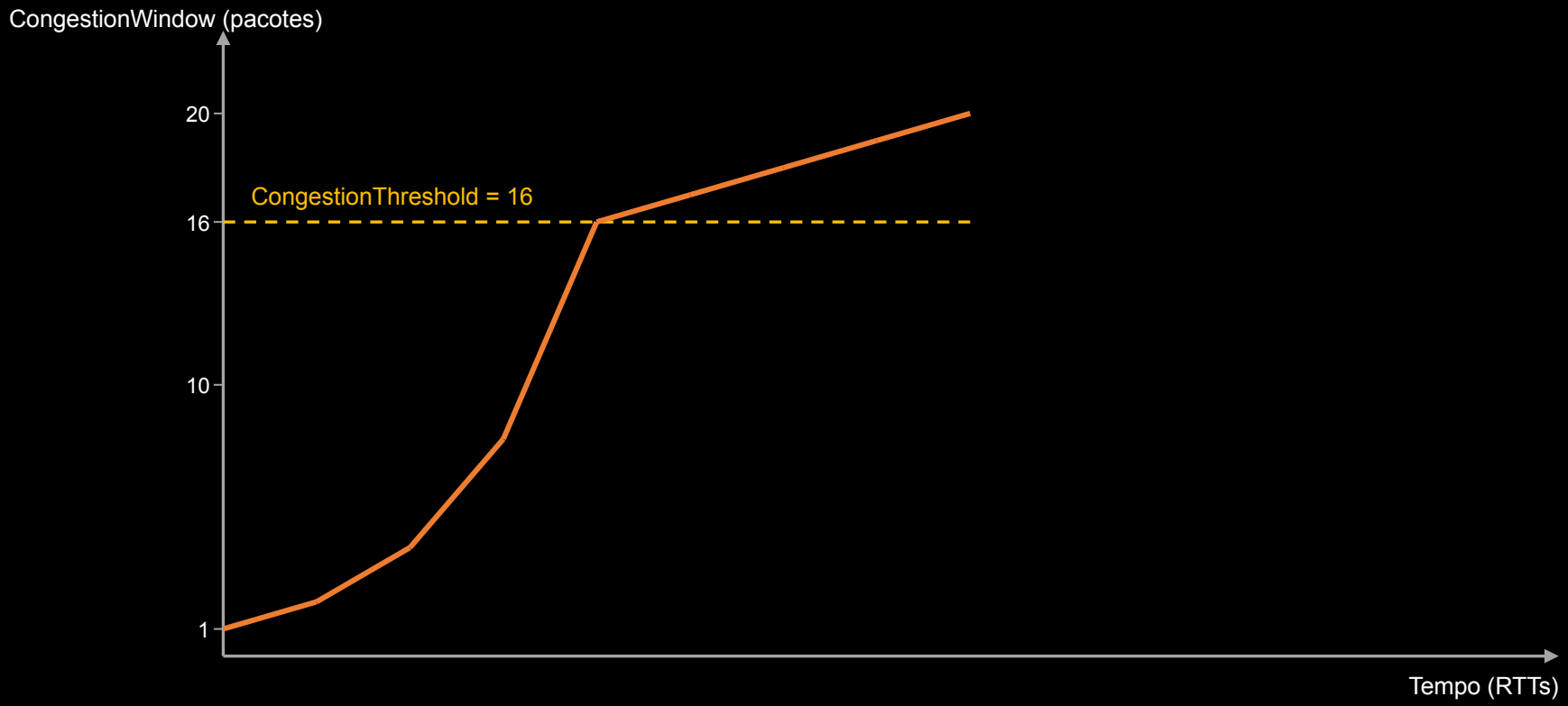
Fast Recovery

- **ACKs duplicados significam que os pacotes continuam a passar:** a ligação ainda não esvaziou, por isso os ACKs ainda conseguem marcar o ritmo de novos envios.

- **Por isso não há necessidade de recomeçar a partir de um pacote:** reduzir o CongestionWindow para metade e continuar com aumento aditivo.

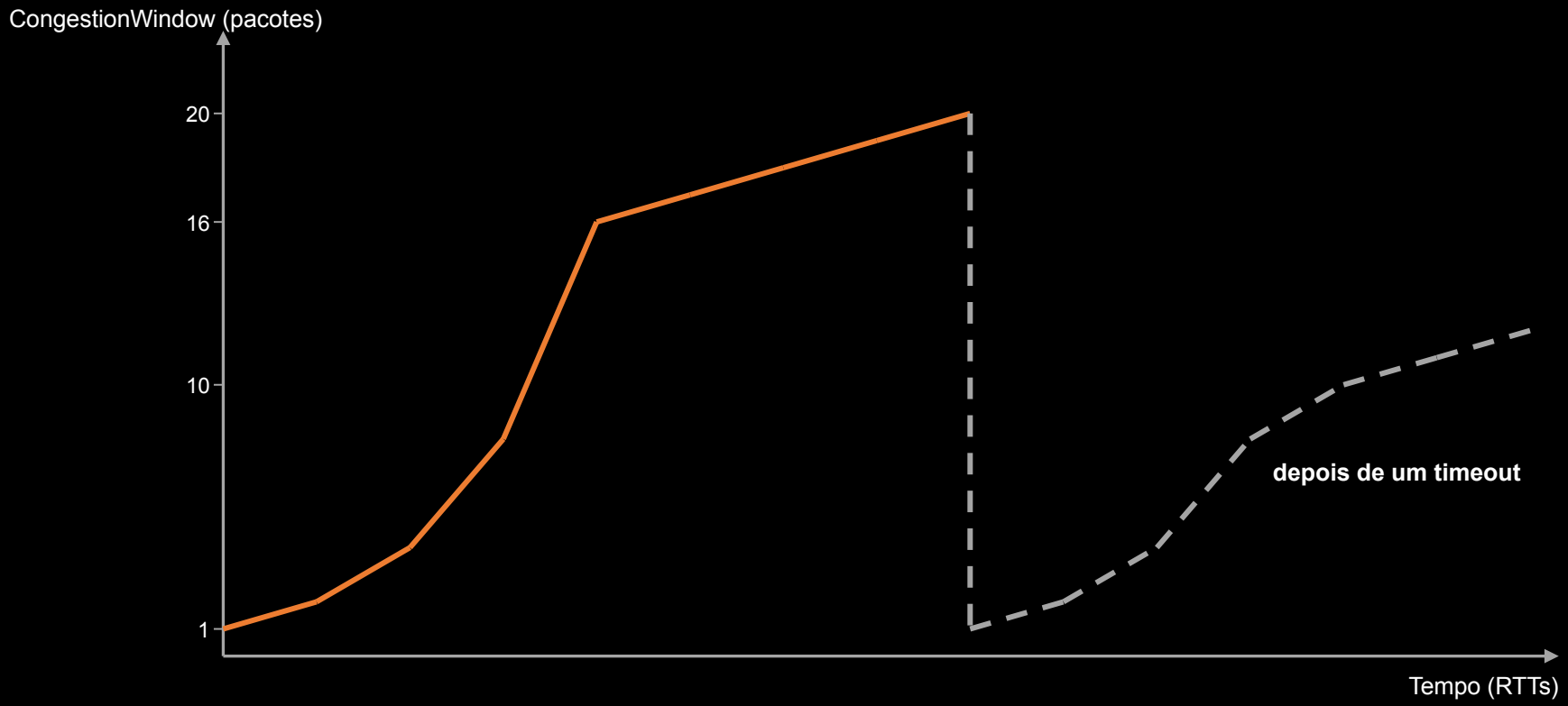
- **O slow start é apenas usado em apenas duas situações:** no início de uma ligação, e depois de um timeout real.

Timeout vs. Fast Recovery



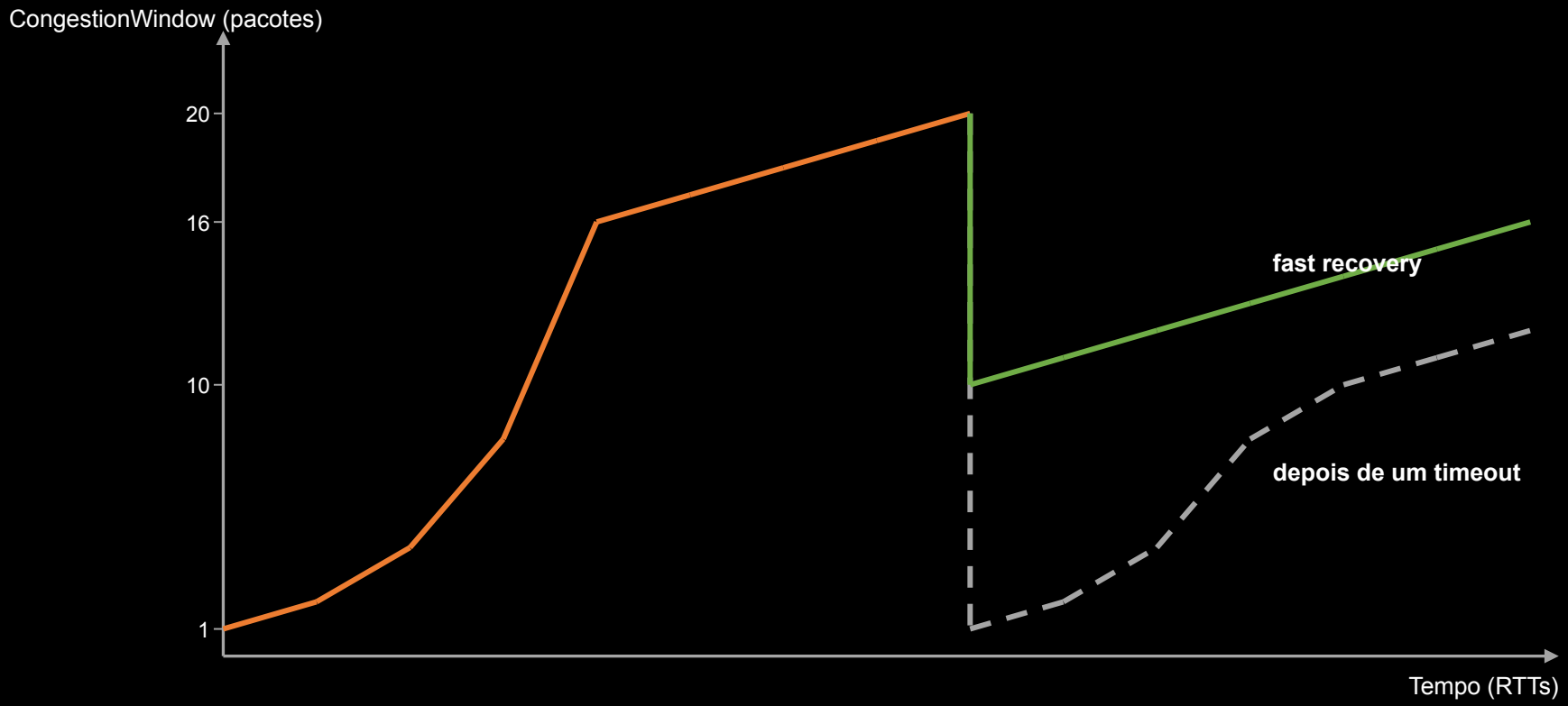
Até ao momento em que uma perda é detetada, com uma janela de 20…

Timeout vs. Fast Recovery



Detetada por um timeout: de volta a um pacote, slow start outra vez.

Timeout vs. Fast Recovery



Detetada por três ACKs duplicados: reduzir para metade e continuar — fast recovery.

Dois Sinais, Duas Reações

Conclusão:
um timeout significa que a ligação esvaziou, por isso o TCP recomeça com cuidado a partir de um pacote; três ACKs duplicados significam que os dados continuam a fluir, por isso reduzir a janela para metade é suficiente.

O TCP Hoje

- **O mecanismo por baixo não mudou.** Toda a ligação TCP hoje ainda abre com o three-way handshake, ainda usa o AdvertisedWindow para controlo de fluxo, e ainda dá timeout de forma adaptativa na perda — os mecanismos clássicos, ainda a correr em todo o lado.
- **Algumas extensões tratam dos links mais rápidos e variados de hoje.** O Window Scale alarga o AdvertisedWindow de 16 bits para caminhos de alta largura de banda, e o timeout adaptativo sensível à variância (Jacobson/Karels) já se ajusta ao próprio RTT de cada ligação — ambos vistos ainda hoje.
- **Mas nenhum deles toca na verdadeira estimativa que o TCP faz sobre a rede.** O tamanho da janela e o timeout são só números — e se a estimativa do TCP sobre a saturação pudesse ser melhorada, para se manter *network friendly* à medida que as condições variam?

CUBIC: Um Aumento Aditivo Mais Inteligente

- **O que substitui:** o aumento aditivo do AIMD clássico — o +1 pacote por RTT fixo, independentemente do tamanho que a janela já tem ou de há quanto tempo foi a última perda.
- **O que faz em vez disso:** faz crescer a janela em função do tempo desde o último evento de saturação, numa curva cúbica — rápida logo a seguir a uma perda, achatando perto do tamanho de janela que a causou, e sondando mais agressivamente quanto mais tempo as coisas ficarem estáveis. A perda continua a ser o sinal, mas o CUBIC recua menos: para 70% da janela, não 50%.
- **Onde vive:** o algoritmo de controlo de saturação por omissão no Linux desde 2006 (substituindo uma variante anterior, o BIC) — a maioria dos servidores, e os telemóveis Android, o correm hoje.

BBR: Um Tipo Diferente de Estimativa

- **O que substitui:** a própria ideia de que a perda é o sinal. O TCP clássico, e o CUBIC, só recuam depois de um pacote já ter sido descartado.
- **O que faz em vez disso:** BBR — Bottleneck Bandwidth and RTT (largura de banda do gargalo e RTT) — lê ambos diretamente dos ACKs: a taxa de entrega mais alta recente estima a largura de banda, e o RTT mais baixo recente estima o atraso de propagação real, já que o queueing effect só permite aumentar o RTT. Corta brevemente o seu ritmo, a cada 10 segundos, para atualizar esse mínimo, e distribui os pacotes a essa largura de banda, mantendo cerca de uma largura de banda×RTT em trânsito — reagindo antes de as filas transbordarem, sem precisar de perda para encontrar o limite.
- **Onde vive:** desenvolvido na Google e usado desde 2016 no seu próprio tráfego (incluindo o YouTube e o seu transporte QUIC); disponível em qualquer Linux desde o kernel 4.9, embora o CUBIC continue a ser o padrão da maioria das distribuições.

Resumo (1/2)

- Uma janela zero é reaberta por Sondas de Janela Zero; as opções Window Scale e Timestamp alargam a janela de 16 bits e o espaço de sequência de 32 bits do TCP para caberem em links rápidos.
- A Silly Window Syndrome é evitada dos dois lados: os recetores não anunciam janelas minúsculas, e o algoritmo de Nagle faz os emissores esperarem por um ACK antes de enviar segmentos pequenos enquanto houver dados por confirmar.
- O timeout de retransmissão adapta-se ao RTT medido e à sua variância (Jacobson/Karels), nunca amostra segmentos retransmitidos (Karn/Partridge), e recua exponencialmente em perdas repetidas.

Resumo (2/2)

- O controlo de saturação protege a rede, não o recetor: o emissor mantém um CongestionWindow e nunca envia mais do que MIN(CongestionWindow, AdvertisedWindow).
- O AIMD reduz a janela para metade numa perda e aumenta-a um pacote por RTT (o dente de serra); o slow start duplica-a a cada RTT, a partir de um pacote até ao CongestionThreshold.
- Três ACKs duplicados despoletam o fast retransmit e o fast recovery (reduzir para metade e continuar); só um timeout manda o TCP de volta ao slow start.

Bibliografia

- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Capítulo 5, Secção 5.2 — Reliable Byte Stream (TCP): 5.2.4 (Sliding Window Revisited), 5.2.5 (Triggering Transmission), 5.2.6 (Adaptive Retransmission) e 5.2.8 (TCP Extensions).
- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Capítulo 6, Secção 6.3 — TCP Congestion Control: 6.3.1 (Additive Increase/Multiplicative Decrease), 6.3.2 (Slow Start), 6.3.3 (Fast Retransmit and Fast Recovery) e 6.3.4 (TCP CUBIC); Secção 6.4.2 (Source-Based Approaches, incluindo TCP BBR).

Bibliografia (cont.)

- Braden, R. (Ed.) (1989). Requirements for Internet Hosts — Communication Layers. RFC 1122 (Secção 4.2.3.6, TCP Keep-Alives).

©2026 NOVA FCT